

Exponential time-differencing with embedded Runge–Kutta adaptive step control



P. Whalen*, M. Brio, J.V. Moloney

Department of Mathematics, University of Arizona, United States

ARTICLE INFO

Article history:

Received 26 November 2013

Received in revised form 27 May 2014

Accepted 30 September 2014

Available online 7 October 2014

Keywords:

ETD

Exponential time-differencing

KdV

Burgers

NLS

Embedded Runge–Kutta

ABSTRACT

We have presented the first embedded Runge–Kutta exponential time-differencing (RKETD) methods of fourth order with third order embedding and fifth order with third order embedding for non-Rosenbrock type nonlinear systems. A procedure for constructing RKETD methods that accounts for both order conditions and stability is outlined. In our stability analysis, the fast time scale is represented by a full linear operator in contrast to particular scalar cases considered before. An effective time-stepping strategy based on reducing both ETD function evaluations and rejected steps is described. Comparisons of performance with adaptive-stepping integrating factor (IF) are carried out on a set of canonical partial differential equations: the shock-fronts of Burgers equation, interacting KdV solitons, KS controlled chaos, and critical collapse of two-dimensional NLS.

© 2014 Elsevier Inc. All rights reserved.

1. Introduction

The primary objective of this paper is to introduce, compare, and analyze several embedded Runge–Kutta (RK) adaptive step methods for the solution of partial differential equations (PDEs) of the form

$$u_t = \mathcal{L}u + \mathcal{N}(u, t), \quad (1)$$

where $\mathcal{L}$ and $\mathcal{N}$ are linear and nonlinear operators, respectively. Often the linear operator $\mathcal{L}$ is either stiff or highly oscillatory and hence standard explicit RK solvers are unfit for numerical simulation. Appropriate computing techniques for solving equations of the type described include exponential time-differencing (ETD) [1], implicit-explicit [2], split-step, integrating factor (IF) [3], and slider algorithms [4]. In the case of fourth order time-stepping, Kassam and Trefethen [5] make a convincing case that ETD is the fastest and most generally applicable method among the ones we listed.

Fourth order Runge–Kutta exponential time differencing (RKETD) was originally developed by Cox and Matthews [1] who needed an improved method for solving stiff PDEs with six spatial derivatives, the general rule being the more derivatives, the stiffer the equation. Krogstad [6] is credited with improving Cox and Matthews' method with another fourth order ETD having slightly better convergence and stability. A major theoretical contribution came from Hochbruck and Ostermann [7] who derived stiff order conditions which are stronger than classical order conditions and arguably more important. We have singled out the papers we found the most useful in our study but the field of exponential integrators is broad and we refer those interested to several thorough reviews [8,9].

One drawback of currently available RKETD methods is the lack of embedded error control. Having adaptive capability is crucial in fields such as optical filamentation where laser pulses undergo critical collapse focusing followed by ionization

* Corresponding author.

defocusing [10]. The underlying length scales in such problems can vary significantly as the pulse propagates. Certain PDE models such as Burgers equation can exhibit steepening as a shock front develops [11]. Relatively small steps are needed near the shock but not far away from it. Obviously there are plenty of other examples in which adaptive time-stepping is desirable.

Kassam and Trefethen [5] concluded that ETD would be costly to extend to variable time-stepping while Minchev and Wright [8] suggested that variable time-stepping could be cheap for periodic systems where $\mathcal{L}$ is a diagonal operator. More recently, Hochbruck et al. constructed several embedded RK exponential Rosenbrock integrators which were shown to be efficient even for non-diagonal problems [12]. In this paper we introduce two embedded RKETD methods and illustrate their efficiency and stability. The adaptive step methods we discuss in this paper are effective in relatively extreme test cases but generally not as effective as one might think. We also show examples where our embedded RKETD methods are slower than their respective constant step embeddings.

For small scale non-diagonal problems, ETD functions are calculated using contour integrations [5] or scaling and squaring algorithms [13,14]. We demonstrate in Section 5.4 that embedded RKETD methods are not well-suited for non-diagonal systems due to how expensive these ETD evaluation costs are relative to RK stage updates. For large scale non-diagonal systems, several efficient exponential integration methods [15,16] have been designed using Krylov subspace methods [17,18]. These types of integrators can be used effectively with an adaptive step for certain non-diagonal problems [12].

2. Constant step Runge–Kutta methods for stiff systems

Before introducing the embedded RK methods we summarize some constant step algorithms for solving systems in the form of Eq. (1). If we group the right-hand side linear and nonlinear responses together

$$F(u, t) = \mathcal{L}u + \mathcal{N}(u, t),$$

then an s -stage explicit RK scheme for solving this system is given by

$$u_{n+1} = u_n + h \sum_{i=1}^s b_i F(k_i, t_n + c_i h),$$

$$k_i = u_n + h \sum_{j=1}^{i-1} a_{ij} F(k_j, t_n + c_j h), \quad i = 1, 2, \dots, s,$$

where an RK Butcher tableau [19] contains the method coefficients

0					
c_2	a_{21}				
c_3	a_{31}	a_{32}			
$\vdots$	$\vdots$		$\ddots$		
c_s	a_{s1}	a_{s2}	$\dots$	$a_{s,s-1}$	
	b_1	b_2	$\dots$	b_{s-1}	b_s

Integrating factor [3] separates out the fast linear operator $\mathcal{L}$ through a change of variable to $v = e^{-t\mathcal{L}}u$ such that

$$v_t = e^{-t\mathcal{L}} \mathcal{N}(e^{t\mathcal{L}}v). \quad (2)$$

Substitution of the right-hand side $e^{-t\mathcal{L}} \mathcal{N}(e^{t\mathcal{L}}v)$ into the RK formulation yields

$$u_{n+1} = e^{h\mathcal{L}}u_n + h \sum_{i=1}^s b_i e^{h\beta_i \mathcal{L}} \mathcal{N}(k_i),$$

$$k_i = e^{h c_i \mathcal{L}}u_n + h \sum_{j=1}^{i-1} a_{ij} e^{h\alpha_{ij} \mathcal{L}} \mathcal{N}(k_j), \quad i = 1, 2, \dots, s, \quad (3)$$

where $\beta_i = 1 - c_i$, $\alpha_{ij} = c_i - c_j$, and $\mathcal{N}(k_i)$ is short-hand for $\mathcal{N}(k_i, t_n + c_i h)$.

It would seem that any constant step RK can be converted into a Runge–Kutta integrating factor (RKIF) method but we do mention one caveat. If $\mathcal{L}$ has an eigenmode with a negative real component and any one of the α_{ij} is negative, then the numerical method will have an exponential growth term not present in the non-IF form. For instance, the embedded method of Fehlberg's RK45 [20] has $\alpha_{65} = -1/2$ and hence the method will not work on PDE's such as Kuramoto–Sivashinsky, Fisher–KPP, and Swift–Hohenberg, among others with negative components in their respective linear eigenmodes. Many popular RK methods have $\alpha_{ij} \geq 0$ for all (i, j) pairs and their corresponding IF schemes do not suffer from this instability.

By replacing $b_i e^{h\beta_i \mathcal{L}}$ and $a_{ij} e^{h\alpha_{ij} \mathcal{L}}$ in Eq. (3) with the more general functions $b_i(h\mathcal{L})$ and $a_{ij}(h\mathcal{L})$, we have the form of an RKETD method

$$\begin{aligned}
u_{n+1} &= e^{h\mathcal{L}} u_n + h \sum_{i=1}^s b_i(h\mathcal{L}) \mathcal{N}(k_i), \\
k_i &= e^{hc_i\mathcal{L}} u_n + h \sum_{j=1}^{i-1} a_{ij}(h\mathcal{L}) \mathcal{N}(k_j), \quad i = 1, 2, \dots, s.
\end{aligned} \tag{4}$$

The difference between ETD and IF lies in how the $b_i(h\mathcal{L})$ and $a_{ij}(h\mathcal{L})$ functions are selected. Formally, we may integrate Eq. (1) from any time t_n to $t_{n+1} = t_n + h$ such that

$$u(t_{n+1}) = e^{h\mathcal{L}} u(t_n) + \int_0^h e^{(h-\tau)\mathcal{L}} \mathcal{N}(u(t_n + \tau), t_n + \tau) d\tau. \tag{5}$$

Integrating factor amounts to applying quadrature rules to the full integral in Eq. (5) whereas the main idea behind ETD methods is to approximate $\mathcal{N}$ in the integrand with a collocation method and then integrate exactly. When the $\mathcal{N}$ function is expanded in a Taylor series polynomial we have

$$u(t_{n+1}) = e^{h\mathcal{L}} u(t_n) + \sum_{r=1}^{\infty} \frac{h^r}{r!} \psi_r(h\mathcal{L}) D^{(r-1)} \mathcal{N}, \tag{6}$$

where the ψ functions are defined as

$$\psi_r(z) = r \int_0^1 e^{(1-\theta)z} \theta^{r-1} d\theta, \quad r = 1, 2, 3, \dots \tag{7}$$

Local truncation errors of the RKETD method and its substages are given by

$$l_{n+1}(h\mathcal{L}) = h \left[\sum_{i=1}^s b_i(h\mathcal{L}) \mathcal{N}(k_i) - \sum_{r=1}^{\infty} \frac{h^{r-1}}{r!} \psi_r(h\mathcal{L}) D^{(r-1)} \mathcal{N} \right], \tag{8}$$

$$l_{n+1,i}(h\mathcal{L}) = h \left[\sum_{j=1}^{i-1} a_{ij}(h\mathcal{L}) \mathcal{N}(k_j) - c_i \sum_{r=1}^{\infty} \frac{(c_i h)^{r-1}}{r!} \psi_r(c_i h\mathcal{L}) D^{(r-1)} \mathcal{N} \right]. \tag{9}$$

When $h\mathcal{L} \rightarrow 0$, the local truncation error reduces to $l_{n+1}(0)$ which is commonly used in standard Runge–Kutta order theory [19]. We also note that as $h\mathcal{L} \rightarrow 0$, the RKETD method reduces to an underlying Runge–Kutta with coefficients $a_{ij}(0)$ and $b_i(0)$. Order conditions for RKETD, which are somewhat more complicated than for standard Runge–Kutta, have been derived using B-series and rooted tree analysis [21]. When used alone, these order conditions can produce unsatisfactory methods with poor convergence and stability. Stiff order conditions [7] are stronger and typically give better results, therefore one should favor these conditions or some combination of stiff order and non-stiff order. Exponential integrators can experience order reduction from non-stiff order down to stiff order and therefore the robustness of the integrator also depends on stiff order. Whether order reduction occurs, how much occurs, and for which step sizes it occurs are largely problem dependent questions.

Our procedure for constructing RKETD methods takes into account stiff order, non-stiff order, and stability. We illustrate our procedure through an example of how to build a six stage, fifth order, stiff order three scheme using Mathematica. First, we used the following stiff order conditions in the substages

$$\sum_{j=1}^{i-1} a_{ij}(c_i h\mathcal{L}) = c_i \psi_1(c_i h\mathcal{L}), \quad \sum_{j=2}^{i-1} a_{ij}(c_i h\mathcal{L}) c_j = c_i^2 \frac{\psi_2(c_i h\mathcal{L})}{2}, \tag{10}$$

where a_{21} satisfies the first condition, and the a_{ij} in substages k_3, k_4, k_5 , and k_6 are chosen to satisfy both conditions. Stiff order conditions are then applied to the final stage

$$\sum_{i=1}^6 b_i(h\mathcal{L}) = \psi_1(h\mathcal{L}), \quad \sum_{i=2}^6 b_i(h\mathcal{L}) c_i = \frac{\psi_2(h\mathcal{L})}{2}, \quad \sum_{i=2}^6 b_i(h\mathcal{L}) c_i^2 = \frac{\psi_3(h\mathcal{L})}{3}. \tag{11}$$

This yields a family of stiff order three methods. There are quite a few degrees of freedom available when fulfilling the stiff order requirements, in particular we decided to express $a_{21}, a_{31}, a_{32}, a_{41}, a_{43}, a_{51}, a_{54}, a_{61}, a_{65}, b_1, b_2$, and b_6 in terms of the unknown functions $a_{42}, a_{52}, a_{53}, a_{62}, a_{63}, a_{64}, b_3, b_4$, and b_5 . The remaining unspecified c_i values and a_{ij} functions are then chosen to satisfy fifth order conditions of the underlying Runge–Kutta method. To this end we expand the a_{ij} in a superposition of ψ functions: $a_{42} = a_{421} \psi_1(c_4 h\mathcal{L}) + a_{422} \psi_2(c_4 h\mathcal{L})$, $a_{5j} = a_{5j1} \psi_1(c_5 h\mathcal{L}) + a_{5j2} \psi_2(c_5 h\mathcal{L})$ for $j = 2, 3$,

$a_{6j} = a_{6j1}\psi_1(c_6 h\mathcal{L}) + a_{6j2}\psi_2(c_6 h\mathcal{L})$ for $j = 2, 3, 4$, and $b_j = b_{j1}\psi_1(h\mathcal{L}) + b_{j2}\psi_2(h\mathcal{L}) + b_{j3}\psi_3(h\mathcal{L})$ for $j = 3, 4, 5$, and choose $c_2 = 1/4$, $c_3 = 1/4$, $c_4 = 1/2$, $c_5 = 3/4$ and $c_6 = 1$. The underlying Runge–Kutta order conditions are met with

$$\begin{aligned} a_{521} &= \frac{1}{8}(-3 - 6a_{421} - 6a_{422} - 8a_{522}), & a_{531} &= \frac{1}{8}(3 + 6a_{421} + 6a_{422} - 8a_{532}), \\ a_{621} &= \frac{1}{7}(8 + 12a_{421} + 12a_{422} - 7a_{622}), & a_{631} &= \frac{1}{7}(6 - 12a_{421} - 12a_{422} - 7a_{632}), \\ a_{641} &= \frac{1}{7}(-12 - 7a_{642}), & b_{31} &= \frac{1}{45}(16 - 45b_{32} - 45b_{33}), \\ b_{41} &= \frac{1}{15}(2 - 15b_{42} - 15b_{43}), & b_{51} &= \frac{1}{45}(16 - 45b_{52} - 45b_{53}). \end{aligned}$$

The remaining non-stiff fifth order conditions are satisfied using Mathematica [22],

$$\begin{aligned} a_{642} &= f_1(a_{421}, a_{422}, a_{522}, a_{532}, a_{622}, a_{632}), & b_{32} &= \frac{1}{30}(-32 - 45b_{33} + 30b_{52} + 45b_{53}), \\ b_{42} &= \frac{1}{20}(16 - 30b_{43} - 30b_{52} - 45b_{53}), & b_{52} &= f_2(a_{421}, a_{422}, a_{522}, a_{532}, a_{622}, a_{632}, b_{33}, b_{43}), \\ b_{53} &= \frac{1}{9}(-16 - 9b_{33} - 12b_{43}), \end{aligned}$$

where f_1 and f_2 are functions of a_{421} , a_{422} , a_{522} , a_{532} , a_{622} , a_{632} , b_{33} , and b_{43} . Free parameters are then used to maximize convergence rates and tune the stability region. We note that this last procedure is a multi-objective optimization problem since increasing the speed of the method typically comes at the cost of a smaller or less desirable stability region; similarly, obtaining the stability region desired will likely result in a slower than optimal method. It's worth mentioning here that RKETD stability regions are not the same as in standard RK theory and can only provide an indication of stability [1,6]. However, practical experience suggests this indication of stability is often reflective of actual stability and therefore we utilize it in the design process.

From the family of methods we have just described, we choose the remaining free parameters based on stability. Part of our selection of the free variables ensured that the integrator was stable for PDE's such as KdV which have purely imaginary linear and nonlinear modes; the corresponding stability region was optimized. We also used the degrees of freedom to make large regions of stability appropriate for a variety of PDE's with both real and imaginary linear modes as well as real and complex nonlinear modes. One set of free parameters that met our requirements was $a_{421} = a_{422} = a_{522} = a_{532} = a_{622} = 0$, $a_{632} = -\frac{87}{28}$, $b_{33} = -\frac{1}{9}$, and $b_{43} = -\frac{6}{5}$. This fifth order integrator is discussed in more detail, with a third order embedding, in Section 3.2.

We conclude this section with a few comments on the procedure we outlined. First, the procedure is not unique and other procedures could be used to build stiff order three, non-stiff order five, stable RKETD integrators. In particular, we could have simply used non-stiff order conditions for orders four and five or we could have skipped the underlying RK step and used Mathematica for all non-stiff order conditions. We preferred to avoid power series expansions of the a_{ij} and b_i functions as occurs in non-stiff order theory. Partly this was for convenience and partly because it is more natural for the a_{ij} and b_i to be expressed in superpositions of ψ functions. Mathematica can be used to meet all non-stiff order requirements but the computation time increases with the number of free parameters. By solving the underlying RK order conditions first, we eliminate some of these free parameters and the remaining non-stiff conditions can be handled quicker. We also note that while tables of order conditions up to non-stiff order four are readily available [21,7], higher orders are not. Tables for order conditions of underlying RK methods are available for orders six and higher [19,23]. Further discussion of how the fifth order ETD scheme is designed along with a corresponding Mathematica script is included in Appendix B.

3. Embedded Runge–Kutta methods for stiff systems

In this section we introduce several embedded Runge–Kutta integrating factor and exponential time-differencing schemes. First, we discuss embedded error control and adaptive time-stepping strategies with a particular emphasis on costs associated with ETD function evaluations. This is followed by the introduction of two adaptive RKETD methods: a fourth order scheme with third order embedding based on Krogstad's method, and a fifth order method with third order embedding that has slightly better speed and stability. We follow up with a subsection on RKIF schemes we use for comparison to RKETD. Lastly, adaptive stepping is illustrated in four PDE examples.

3.1. Error control and adaptive stepping strategies for embedded Runge–Kutta

Our embedded error control is largely based on existing continuous Runge–Kutta theory. In particular, we modified a standard local-error-per-step with local extrapolation strategy used in continuous RK [24] such that it would apply for our purposes. For IF and ETD methods, a Runge–Kutta s -stage order $(q - 1, p)$ formula pair is

$$\bar{u}_{n+1} = e^{h\mathcal{L}} u_n + h \sum_{i=1}^s \bar{b}_i(h\mathcal{L}) \mathcal{N}(k_i), \quad u_{n+1} = e^{h\mathcal{L}} u_n + h \sum_{i=1}^s b_i(h\mathcal{L}) \mathcal{N}(k_i),$$

where $k_i = e^{hc_i\mathcal{L}} u_n + h \sum_{j=1}^{i-1} a_{ij}(h\mathcal{L}) \mathcal{N}(k_j)$, $i = 1, 2, \dots, s$, and $q \leq p$. Consider u_{ex} to be the exact solution to Eq. (1) at t_{n+1} , then it follows that $\bar{u}_{n+1} = u_{ex} + \bar{e}_{n+1}h^q$ and $u_{n+1} = u_{ex} + O(h^{p+1})$. Therefore the embedding estimate $E_{n+1} = \bar{u}_{n+1} - u_{n+1}$ satisfies

$$E_{n+1} = h \sum_{i=1}^s [\bar{b}_i(h\mathcal{L}) - b_i(h\mathcal{L})] = \bar{e}_{n+1}h^q + O(h^{q+1}).$$

For a given local truncation error tolerance (TOL), we find conditions relating the “optimal” step-size h_{opt} to the error term $\bar{e}_{n+1}$

$$\|\bar{e}_{n+1}\| h^q \approx \|E_{n+1}(h\mathcal{L})\|, \quad \|\bar{e}_{n+1}\| h_{opt}^q \leq TOL.$$

Solving for h_{opt} , we have

$$h_{opt} \leq h \left[\frac{TOL}{\|E_{n+1}(h\mathcal{L})\|} \right]^{\frac{1}{q}} = \nu h \left[\frac{TOL}{\|E_{n+1}(h\mathcal{L})\|} \right]^{\frac{1}{q}}, \quad (12)$$

where $0 < \nu < 1$ is a safety factor. All tested methods use the Euclidean $\|\cdot\|_2$ norm with an error tolerance $TOL = \epsilon_{rel} \|u_{n+1}\|_2$. We propagate PDE's and evaluate error in the spectral space and hence Parseval's theorem on preservation of L_2 norms suggests the Euclidean norm would be appropriate; we also tested other norms including the sup norm $\|\cdot\|_\infty$ and speeds tended to be marginally faster using $\|\cdot\|_2$.

In constructing adaptive ETD it is important to consider the cost of evaluating the coefficient functions $a_{ij}(h\mathcal{L})$ and $b_i(h\mathcal{L})$ which must be updated whenever h is increased or decreased. We will use a contour integral to evaluate the functions as proposed in [5]. Even for diagonal (periodic) problems we find that this can be an expensive proposition unless done efficiently. The case when $\mathcal{L}$ is non-diagonal is not considered until Section 5.4. For diagonal $\mathcal{L}$ we use two observations to improve speed: (1) For each scalar mode c of the linear operator $\mathcal{L}$, it is only necessary to contour integrate the ETD functions when hc is sufficiently small, and (2) h_{opt} can be far from optimal when step size adjustments are costly. Based on observation (1), we select $hc = 0.01$ as our cutoff for computing ETD functions using a contour integral. For constant step solvers this consideration is not essential since the contour integrals are evaluated only once. From observation (2), we first let

$$s = \nu \left[\frac{TOL}{\|E_{n+1}(h\mathcal{L})\|} \right]^{\frac{1}{q}},$$

such that $h_{opt} = s * h$. Then we define a “restrained” step size $h_r = \mu * h$ where

$$\mu = \begin{cases} \mu_{\min} & s \leq \mu_{\min} \\ s & \mu_{\min} \leq s < \mu_c \\ \mu_c & \mu_c \leq s < 1 \\ 1 & 1 \leq s < \mu_e \\ s & \mu_e \leq s < \mu_{\max} \\ \mu_{\max} & s > \mu_{\max}. \end{cases}$$

The parameters $\mu_{\min}$ and $\mu_{\max}$ are standard embedded RK parameters that contribute to stability. The parameters μ_c and μ_e are selected to keep $\mu = 1$ for a large percentage of the propagation length while still maintaining adaptive capabilities for when they are needed. We found that $\mu_c = 0.85$ and $\mu_e = 1.25$ work well for a broad range of ϵ_{rel} tolerance levels and a variety of nonlinear PDE models and hence use these as default values. However, the μ_c and μ_e that yield the best results depend on both ϵ_{rel} and the specific PDE problem at hand.

If $\mu < 1$, the current step failed and u_{n+1} is recomputed with step size h_r . If $\mu = 1$, u_{n+1} is accepted with the same step size as in the current step and it is unnecessary to update the RK functions. We also accept u_{n+1} when $\mu > 1$ but with a new step size h_r . Local extrapolation is used such that u is updated with the higher order embedded method u_{n+1} . While restrained adapting was originally created to ensure a low number of ETD function evaluations, we have found that it also keeps the number of rejected steps to a minimum. Since the $\mathcal{N}(k_i)$ stage evaluations can be expensive, adaptive stepping with h_{opt} is often inefficient even for embedded RKIF, hence we use the restrained adapting strategy on all tested methods.

3.2. Embedded RKETD methods

Arguably the best explicit fourth order Runge–Kutta exponential time differencing scheme is from Krogstad (ETDRK4-B) [6]. With this in mind our first goal was to extend Krogstad's method to adaptive stepping. To do this we used the first-same-as-last (FSAL) principle [23] and let $a_{5i} = b_i$ for $i = 1, 2, 3, 4$ while keeping Krogstad's substages k_1, k_2, k_3 , and k_4 .

Table 1

ETD34 embedded Runge–Kutta ETD. Both the fourth order and third order methods are stiff order three. The ψ functions for stage i are evaluated as $\psi_j = \psi_j(c_i h \mathcal{L})$.

0					
$\frac{1}{2}$	$\frac{1}{2}\psi_1$				
$\frac{1}{2}$	$\frac{1}{2}\psi_1 - \frac{1}{2}\psi_2$	$\frac{1}{2}\psi_2$			
1	$\psi_1 - \psi_2$	0	ψ_2		
1	$\psi_1 - \frac{3}{2}\psi_2 + \frac{2}{3}\psi_3$	$\psi_2 - \frac{2}{3}\psi_3$	$\psi_2 - \frac{2}{3}\psi_3$	$-\frac{1}{2}\psi_2 + \frac{2}{3}\psi_3$	
	$\psi_1 - \frac{3}{2}\psi_2 + \frac{2}{3}\psi_3$	$\psi_2 - \frac{2}{3}\psi_3$	$\psi_2 - \frac{2}{3}\psi_3$	$-\frac{1}{2}\psi_2 + \frac{2}{3}\psi_3$	
	$\psi_1 - \frac{3}{2}\psi_2 + \frac{2}{3}\psi_3$	$\psi_2 - \frac{2}{3}\psi_3$	$\psi_2 - \frac{2}{3}\psi_3$	0	$-\frac{1}{2}\psi_2 + \frac{2}{3}\psi_3$

Table 2

ETD35 embedded Runge–Kutta ETD. Both the fifth order and third order methods are stiff order three. The ψ functions for stage i are evaluated as $\psi_j = \psi_j(c_i h \mathcal{L})$.

0						
$\frac{1}{4}$	$\frac{1}{4}\psi_1$					
$\frac{1}{4}$	$\frac{1}{4}\psi_1 - \frac{1}{8}\psi_2$	$\frac{1}{8}\psi_2$				
$\frac{1}{2}$	$\frac{1}{2}\psi_1 - \frac{1}{2}\psi_2$	0	$\frac{1}{2}\psi_2$			
$\frac{3}{4}$	$\frac{3}{4}\psi_1 - \frac{9}{16}\psi_2$	$-\frac{3}{8}\psi_1$	$\frac{3}{8}\psi_1$	$\frac{9}{16}\psi_2$		
1	$-\frac{77}{42}\psi_1 + \frac{59}{42}\psi_2$	$\frac{8}{7}\psi_1$	$\frac{111}{28}\psi_1 - \frac{87}{28}\psi_2$	$-\frac{12}{7}\psi_1$	$-\frac{47}{84}\psi_1 + \frac{143}{84}\psi_2$	
1	b_1	0	b_3	b_4	b_5	b_6
	b_1	0	b_3	b_4	b_5	b_6 0
	$\bar{b}_1$	0	$\bar{b}_3$	$\bar{b}_4$	0	$\bar{b}_6$ 0

$$\begin{aligned}
 b_1 &= \frac{7}{2700}(257\psi_1 - 497\psi_2 + 270\psi_3) \\
 b_3 &= \frac{1}{1350}(1097\psi_1 - 497\psi_2 - 150\psi_3) \\
 b_4 &= -\frac{2}{225}(49\psi_1 - 199\psi_2 + 135\psi_3) \\
 b_5 &= \frac{1}{1350}(-313\psi_1 + 883\psi_2 - 90\psi_3) \\
 b_6 &= \frac{1}{2700}(509\psi_1 - 2129\psi_2 + 1830\psi_3)
 \end{aligned}$$

$$\begin{aligned}
 \bar{b}_1 &= \frac{1}{120}(73\psi_1 - 135\psi_2 + 82\psi_3) \\
 \bar{b}_3 &= \frac{1}{45}(47\psi_1 - 45\psi_2 - 2\psi_3) \\
 \bar{b}_4 &= \frac{1}{60}(-47\psi_1 + 165\psi_2 - 78\psi_3) \\
 \bar{b}_6 &= \frac{1}{360}(47\psi_1 - 225\psi_2 + 238\psi_3)
 \end{aligned}$$

Next, we construct an embedded third order method. The embedded method can be made stiff order three by solving for $\bar{b}_1$, $\bar{b}_2$, $\bar{b}_3$, and $\bar{b}_5$ as functions of the free parameter $\bar{b}_4$. Enforcing $\bar{b}_4 \neq b_4$ guarantees the embedded formulae are not equivalent. The simplest choice for $\bar{b}_4$ is zero and when tested on several numerical initial value problems, the error control was acceptable. The resulting ETD34 scheme is shown in Table 1.

While constructing an embedded method from an existing constant step RKETD was convenient, better embedded methods can be made from scratch using stiff order conditions. In Section 2 we outlined a procedure for building a fifth order, stiff order three RKETD which has slightly better speed and stability than Krogstad's ETD34-B. Initially we tried coupling this method with a fourth order embedding but every variation we tested yielded poor error control. We believe the problem is due to stiff order [7] since the constant step version of the fifth order embedding commonly experiences order reduction. This problem with error control was handled, with mild success, by using a third order embedding. The resulting ETD35 scheme is shown in Table 2. We note that the $\bar{b}_i$ are selected such that $\bar{b}_i - b_i$ is proportional to b_5 . This is done primarily to reduce the number of ETD function evaluations.

3.3. Embedded RKIF methods

One major concern for embedded RKETD is the cost of evaluating the ψ functions each time the step size is adapted. For diagonal problems, integrating factor functions $a_{ij}(h\mathcal{L})$ and $b_i(h\mathcal{L})$ are often significantly cheaper to evaluate than their ETD counterparts. Therefore, it is reasonable to consider that embedded RKIF may run faster than embedded RKETD. For comparison we take the underlying Runge–Kutta of the ETD34 from the previous section and convert it to an integrating factor method (IF34) (see Table 3).

One concern with the IF34 algorithm is that it is built on an underlying Runge–Kutta for ETD34 which was designed to satisfy stiff order conditions and not to have the most efficient error control. Therefore, we also consider an integrating factor algorithm based on Dormand–Prince's [23] fifth order method with embedded fourth order error control (IFDP45). In Section 5.1, Fig. 4, we show that IF34 actually outperforms IFDP45 in some test cases. This is not a complete surprise as the most computationally efficient integrating factor methods have uniformly spaced c_i with repetitions [8].

Table 3

IF34 embedded Runge–Kutta IF. Based on the underlying Runge–Kutta of ETD34. In the exponential functions we have $z = h\mathcal{L}$.

0					
$\frac{1}{2}$	$\frac{1}{2}e^{z/2}$				
$\frac{1}{2}$	0	$\frac{1}{2}$			
1	0	0	$e^{z/2}$		
1	$\frac{1}{6}e^z$	$\frac{1}{3}e^{z/2}$	$\frac{1}{3}e^{z/2}$	$\frac{1}{6}$	
	$\frac{1}{6}e^z$	$\frac{1}{3}e^{z/2}$	$\frac{1}{3}e^{z/2}$	$\frac{1}{6}$	
	$\frac{1}{6}e^z$	$\frac{1}{3}e^{z/2}$	$\frac{1}{3}e^{z/2}$	0	$\frac{1}{6}$

3.4. Illustration of adaptive stepping RKETD

We consider four PDE's, all with periodic boundary conditions. The first three PDE's are KdV, KS, and Burgers, which are canonical equations selected for testing purposes. The fourth PDE is an optical filamentation equation based on NLS. It is propagated longitudinally in z rather than in time which is discretized using a periodic Fourier grid. This equation is a simplified version of what is used in modern optical filamentation codes but it does capture most of the important physics which include self-focusing collapse followed by ionization defocusing [10]. All four PDE's are propagated spectrally with both physical and spectral space field components u and $\hat{u}$ treated as complex quantities. Some computational efficiency is lost when u is a real periodic function (KdV, KS, and Burgers') but treating u as complex allows us to use more general algorithms. The test examples are listed as follows:

1. KdV equation (see [25])

$$u_t = -u_{xxx} - 6uu_x, \quad x \in [-150, 150],$$

$$u(x, t=0) = \sum_{j=1}^5 3A_j^2 \operatorname{sech}\left(\frac{A_j(x-x_j)}{2}\right)^2,$$

with amplitudes corresponding to $A_1 = 0.6$, $A_2 = 0.5$, $A_3 = 0.4$, $A_4 = 0.3$, and $A_5 = 0.2$, and x -axis translations $x_1 = -120$, $x_2 = -90$, $x_3 = -60$, $x_4 = -30$, and $x_5 = 0$. The propagation time runs for $t = 600$. We use a 512-point Fourier spectral discretization for this and the next PDE example.

2. KS equation (see [5]),

$$u_t = -u_{xx} - u_{xxx} - uu_x, \quad x \in [0, 32\pi],$$

$$u(x, t=0) = \cos\left(\frac{x}{16}\right)\left(1 + \sin\left(\frac{x}{16}\right)\right),$$

with a propagation time of $t = 50$.

3. Burgers' equation (see [5]),

$$u_t = \nu u_{xx} - uu_x, \quad x \in [-\pi, \pi],$$

$$u(x, t=0) = \exp(-10 \sin^2(x/2)),$$

where $\nu = 0.0005$, the simulation runs to $t = 1$, and an 8192-point Fourier discretization is utilized.

4. Optical filamentation NLS (see [26])

$$\frac{\partial \mathcal{E}}{\partial z} = \frac{i}{2k_0} \nabla_{\perp}^2 \mathcal{E} - \frac{ik''}{2} \frac{\partial^2 \mathcal{E}}{\partial t^2} + \frac{in_2 \omega_0}{c} |\mathcal{E}|^2 \mathcal{E} + \frac{i\omega_0}{2n_0 c \rho_c} \rho \mathcal{E} - \frac{\beta_K}{2} |\mathcal{E}|^{2K-2} \mathcal{E},$$

$$\frac{\partial \rho}{\partial t} = \sigma_K |\mathcal{E}|^{2K} (\rho_{nt} - \rho),$$

$$\mathcal{E}(r, t, z=0) = \sqrt{I_0} \exp\left(-\frac{r^2}{w_0^2} - \frac{t^2}{t_p^2}\right),$$

where we propagate from $z = 0$ to $z = 10.34$ cm. Many of our parameters are taken directly from [26], in particular we use $\lambda_0 = 800$ nm, $n_0 = 1.334$, $n_2 = 4.1 \times 10^{-16}$ cm²W⁻¹, $k'' = 248$ fs²cm⁻¹, $K = 5$, $\beta_K = 1 \times 10^{-47}$ cm⁷W⁻⁴, $\sigma_K = 1.2 \times 10^{-52}$ s⁻¹cm¹⁰W⁻⁵, and $\rho_{nt} = 6.68 \times 10^{22}$ cm⁻³. However, we did modify our initial pulse profile to increase both the collapse distance and growth in the magnitude of the optical field. This was done by selecting $t_p = 160$ fs, $w_0 = 250$ μ m, and $I_0 = 8.0 \times 10^9$ Wcm⁻². Radially we use a 600-point Hankel spectral discretization [27] with $r \in [0, 4w_0]$, and temporally we use a 1024-point Fourier spectral discretization with $t \in [-6t_p, 6t_p]$. The remaining unspecified parameters can be computed from formulas provided in [26,10].

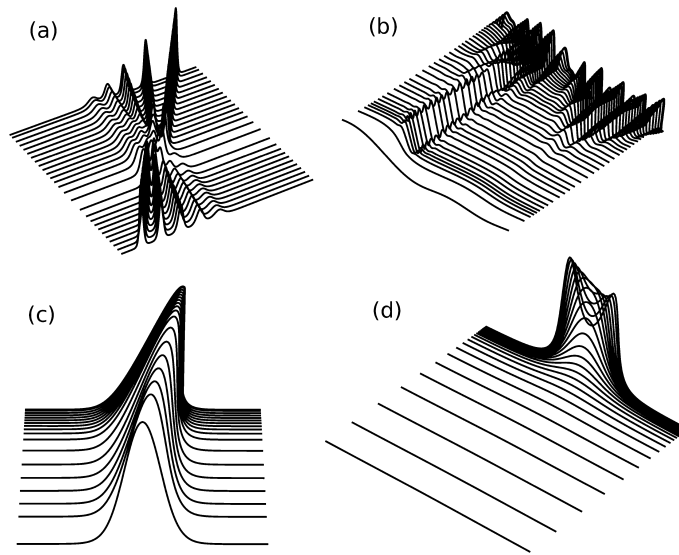


Fig. 1. Illustration of adaptive stepping ETD35 with $\epsilon_{rel} = 1.0 \times 10^{-6}$ on (a) KdV solitons, (b) KS chaos, (c) Burgers' shock fronts, and (d) NLS critical collapse with plasma defocusing and nonlinear absorption regularization mechanisms.

Adaptive stepping ETD35 for the four PDE's is demonstrated in Fig. 1. We used $\epsilon_{rel} = 1.0 \times 10^{-6}$ for our error tolerance level, with $\mu_c = 0.95$ and $\mu_e = 1.05$ selected to allow frequent adapting (not efficient adapting). For the KdV, KS, and Burgers' equations we plot u_n at evenly spaced n . Since an adaptive step is utilized, the profiles in the field evolution will be more closely bunched together in regions of the propagation using a small step and more spread out in regions of the propagation where large steps are used. We also note that the final field profile is shown for each PDE model and so if a small step is taken at the end, this is an indication of the simulation terminating not of the error control reducing the step size. For the optics NLS equation we plot $|\mathcal{E}(r=0, t, z_n)|^2$ at evenly spaced n . This corresponds to the on-axis intensity which is more useful in optical filamentation than $\mathcal{E}(r=0, t, z_n)$.

Adaptive stepping has about the same efficiency as constant stepping for interacting KdV solitons. Initially, a relatively constant step size is taken until the solitons start colliding into each other, at which point the step size increases. After the solitons interact and return back to their initial states the step size decreases and becomes relatively constant again. Since the soliton interaction region is limited and does not generate significant step size changes there is not much available to gain in terms of speed.

The KS example in Fig. 1(b) seems to benefit more from an error controlled step size than KdV. However, Fig. 1 does not tell the whole story since rejected steps tend to be costly. For ETD35, the KS may be the least efficient of the four examples when compared to the constant step embedded fifth order method. In terms of speed, the best μ_c and μ_e parameters tend to favor very little adapting and hence we end up with an optimally quasi constant step solver. Improved error control strategies, or embedded methods which limit rejected $\mathcal{N}$ function evaluations could potentially make embedded RKETD efficient for a broader range of problems, including this one.

We chose a relatively small dissipation value $\nu = 0.0005$ such that strong shock formation occurs in Burgers' test case. Since many more steps are taken near the shock front than far away from it, adaptive stepping is often more efficient than constant stepping. We do note that for certain error tolerances, ETD function costs may negate any gains from reduced nonlinear function evaluations. In Section 5.1 we discuss this in more detail.

The optical filamentation NLS problem benefits the most from embedded error control. The step size is constantly being reduced as the pulse undergoes critical self-focusing collapse [28]. At some stage the magnitude of $\mathcal{E}$ becomes large enough to ionize the medium (water) and during this process energy is lost through nonlinear absorption. After enough energy is removed from the system, dispersion and plasma are strong enough to defocus the beam and the step size gradually increases. We also note that because the PDE is discretized over both a radial and a temporal dimension, each step can be rather costly. This makes minimizing the number of steps taken in a simulation important as the computation time is not insignificant.

4. Stability

The stability analysis of exponential integrators [29,1] begins with linearizing Eq. (1) about a fixed point u_0 . This yields the test equation

$$\dot{v} = \mathcal{L}v + \lambda v,$$

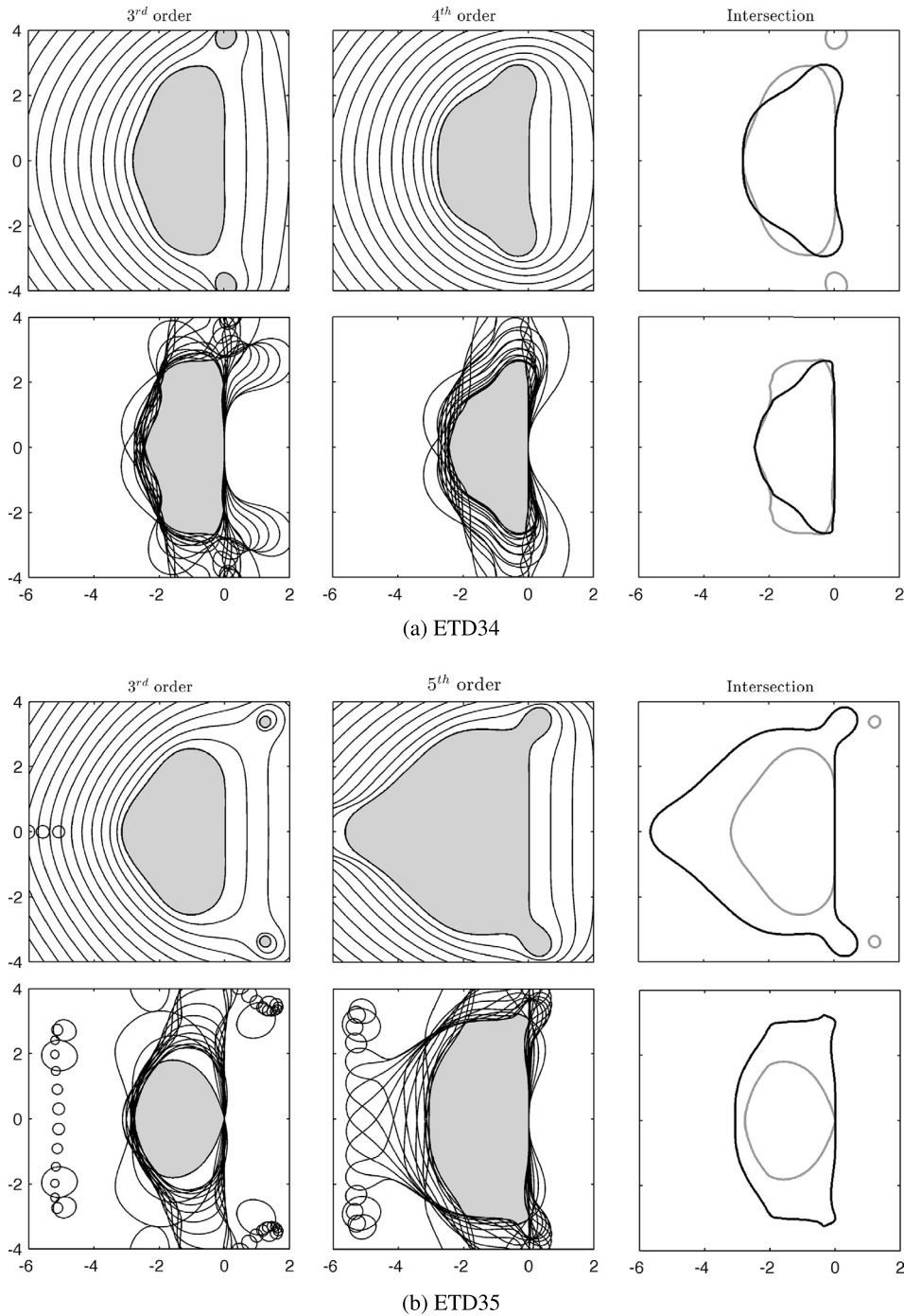


Fig. 2. Stability regions for embedded RKETD methods. The top rows corresponds to $hc \in \mathbb{R}^-$, the bottom rows corresponds to $hc \in [-2\pi i, 2\pi i]$.

where $v = u - u_0$ and $\lambda = \mathcal{N}'(u_0)$. In the linearization, $\mathcal{L}v$ and λv have two different time scales with $\mathcal{L}v$ being stiff or fast and λv being slow. Next, we apply ETD34 and ETD35 to the test equation with $\mathcal{L}$ considered the linear term and λu taken as the nonlinear term. This generates expressions of the form

$$u_{n+1} = G(h\lambda; h\mathcal{L})u_n,$$

where h is a step size and G is an amplification factor. Following Krogstad [6], we use MATLAB's contour function to plot $h\lambda \in \mathbb{C}$ where $|G| = 1$ for several modes c of $\mathcal{L}$ and then examine the intersecting region.

Ideally one would consider values of c encompassing the entire spectrum of $\mathcal{L}$, but this is problem specific and so two general cases are examined instead. The first case is for $hc \in \mathbb{R}^-$ which corresponds to the top rows of the stability plots

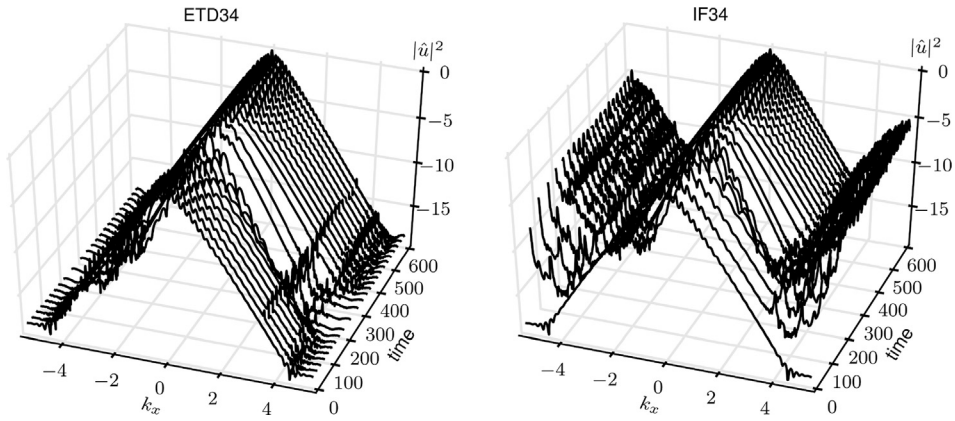


Fig. 3. Evolution of the normalized power spectrum $|\hat{u}|^2$ for interacting KdV solitons in two numerical cases. The larger modes of the system are stable for exponential time differencing (ETD34) but not for integrating factor (IF34).

in Figs. 2(a) and 2(b). The second case is for $hc \in [-2\pi i, 2\pi i]$ which corresponds to the bottom rows of the stability plots in Figs. 2(a) and 2(b). Included in Appendix A is a MATLAB code for computing stability regions of ETD34 for problem specific $\mathcal{L}$ including both diagonal and non-diagonal operators. The script evaluates the spectral radius $\rho(G)(h\lambda)$ for each $h\lambda$ in a subset of the complex plane and plots the contour $|\rho(G)(h\lambda)| = 1$. For diagonal $\mathcal{L}$, the spectral radius procedure is equivalent to looking at intersecting contours as in the rightmost subplots of Fig. 2.

Local extrapolation is used such that the higher order embedded RKETD is taken for the update u_{n+1} , therefore the stability region of the higher order method is the one that dictates the stability of the entire method. In standard embedded RK theory it has been argued that the stability region of the lower order method can be important for step size control [23], however it is not clear whether this translates to embedded RKETD. In testing ETD35, which has rather different third order and fifth order stability regions, we have not observed problems in step size control generated from instabilities in the lower order method. For a given step size, if the higher order RK is stable while the lower order RK is unstable *and* the step size is large enough for the instability to grow significantly, then we could run into problems. We believe this is more likely to happen in linear RK than in nonlinear RKETD as the nonlinearity naturally restricts step sizes, however we stop short of claiming that it cannot happen.

In ETD34, the fourth order stability region corresponds to Krogstad's ETD34-B which is known to have excellent stability for nonlinear PDE's with both real and imaginary linear modes. Fig. 2 shows that the fifth order method in ETD35 has slightly larger regions of stability than Krogstad's fourth order method. As we mentioned in Section 2, the large stability regions of the fifth order method in ETD35 are part of the design of the method.

While we have not plotted stability regions for the IF method, we do mention that generally IF schemes are less stable than their ETD counterparts. The KdV soliton test case from Section 3.4 is an example of a problem which is unstable for IF but stable for ETD. This is demonstrated in Fig. 3 which shows the evolution of spectral power. As a result of instability, the IF34 has an artificial growth of power in the higher order modes of the system.

5. Numerical experiments

In this section we present results from several tests on the performance of IF34, ETD34, IFDP45, and ETD35. Each test is run on the problems in Section 3.4. Our procedure for computing errors in test simulations is similar to the one found in [5]; we estimate the “true” solution of each test problem using a numerical solution with high accuracy and then compute error as the $\|\cdot\|_\infty$ norm of the difference between the test solution and “true” solution divided by the maximum value of the “true” solution. In particular, we used the IFDP45 with extremely small ϵ_{rel} to produce our high-accuracy solutions used in computing relative errors. We note that the relative errors generated by using a high-accuracy constant step ETD method as the “true” solution were nearly identical to those generated by using the high-accuracy IFDP45 as the “true” solution and so the choice of using the IFDP45 is somewhat arbitrary.

For the NLS test case we found the error of the on-axis intensity $|\mathcal{E}(r=0, t, z_n)|^2$ rather than the error in $\mathcal{E}(r, t, z_n)$. This corresponds to the NLS evolution shown in Fig. 1(d). Similar results are obtained when using the intensity over the entire radial grid. All simulations use the adapting parameters $\mu_c = 0.85$ and $\mu_e = 1.25$ which yielded acceptable results for all four PDE's tested over a broad range of error tolerances. We use the same μ_c and μ_e for consistency but do emphasize that performance is dependent on these values and therefore results should not be interpreted as optimal. Our maximum step size increase factor and minimum step size decrease factor were selected to be $\mu_{max} = 2.5$ and $\mu_{min} = 0.4$, respectively. Simulations are based on C++ code compiled and run on a Mac Pro (Early 2009) with two 2.66 GHz Quad-Core Intel Xeon processors. Different implementations, run on different machines, will give different results, though we believe the general trends and conclusions about speed and accuracy of the various methods will be the same.

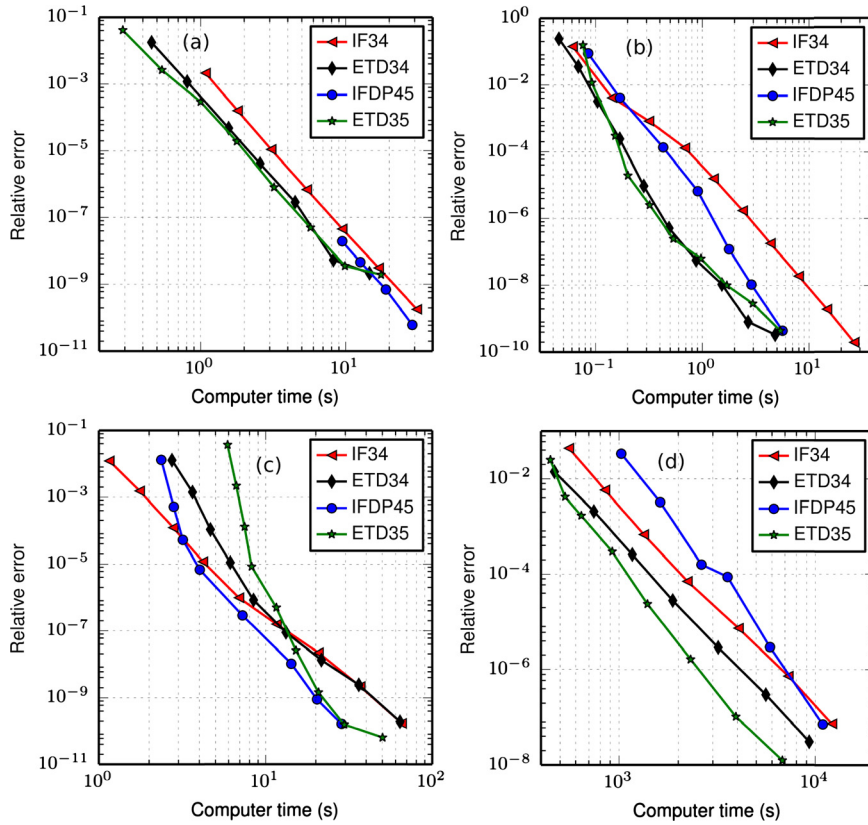


Fig. 4. Efficiency of several embedded RK methods for stiff systems on (a) KdV solitons, (b) KS chaos, (c) Burgers' shock fronts, and (d) NLS critical collapse with plasma defocusing and nonlinear absorption regularization mechanisms.

5.1. Efficiency

The first test we performed compares speed to relative error. Each data point in Fig. 4 corresponds to a different ϵ_{rel} tolerance with an initial step size chosen such that the first step is accepted. None of the methods clearly distinguishes itself as faster than the rest for all test cases but there are certain situations where particular methods work best. The embedded RKETD methods outperformed the RKIF methods for KdV, KS, and NLS test cases with ETD35 being slightly faster than ETD34 for NLS. In the Burgers shock test, IF34 performed the best for weak error tolerances and IFDP45 performed the best for strong error tolerances. There are also situations in which one should clearly avoid certain methods. For instance, both of the IF schemes are unstable in KdV, and hence larger tolerance levels yield poor results.

Practically, speed is more important in the optics NLS equation than in the other examples because the simulations take much longer. In fact, speed is a significant factor in the field of filamentation optics where it is not uncommon for simulations to take days rather than minutes or seconds. Both ETD34 and ETD35 performed favorably vs. IF34 and IFDP45 with simulations typically running 50–100% faster. We note that simulations with relative errors between 10^{-2} and 10^{-4} are arguably the most important since time considerations and limited computer resources make it more efficient to run lots of simulations at low accuracies before settling on a “best result” which can be re-run at higher accuracies. In this important error range, ETD35 performs the best and IFDP45 should be avoided.

For all the tested cases we see the IFDP45 method catching up in speed to the ETD methods as the relative error is reduced. Part of this is due to a saturation of the ETD methods for small ϵ_{rel} tolerances, however we also mention that IFDP45 has the best underlying Runge–Kutta algorithm. As $\epsilon_{rel} \rightarrow 0$, the step sizes become smaller and the performance of the methods should approach the performance of the underlying Runge–Kutta method. Therefore, it should be expected that IFDP45 will perform the best for extremely strong tolerance levels. Of course the reason for using a stiff solver is to avoid very small steps and so the value of this is unclear.

One interesting effect is the interplay between ETD function costs and RK step costs. The RK step costs are primarily due to the nonlinear function evaluations which in some cases dominate the total overall computation time, e.g. the NLS tests. On the other hand, the Burgers and KS tests have a significant percentage of their costs associated with ETD function evaluations; if the parameters μ_c and μ_e are chosen too close to one, more time can be spent evaluating ETD functions than updating the RK stages. This means that adaptive stepping RKETD is limited by ETD function costs in some cases but not

Table 4

ETD35 step update and ETD function evaluation costs.

ϵ_{rel}	Burgers		Relative error	ϵ_{rel}	NLS		Relative error
	Step (s)	ETD (s)			Step (s)	ETD (s)	
1.0×10^{-2}	2.31	3.58	3.7×10^{-2}	7.0×10^{-4}	394.5	51.3	2.5×10^{-2}
1.0×10^{-4}	3.58	3.81	1.3×10^{-4}	1.0×10^{-4}	580.9	59.4	1.7×10^{-3}
1.0×10^{-6}	6.56	4.96	5.0×10^{-7}	1.0×10^{-5}	860.8	61.2	3.0×10^{-4}
1.0×10^{-8}	15.80	4.68	1.4×10^{-9}	1.0×10^{-6}	1334.4	59.8	2.4×10^{-5}
1.0×10^{-10}	47.02	2.59	6.4×10^{-11}	1.0×10^{-7}	2251.3	61.7	1.6×10^{-6}

Table 5

Parallelization of ETD35 function evaluations.

ϵ_{rel}	1 thread (s)	% of n.t.	2 threads (s)	% of n.t.	4 threads (s)	% of n.t.	8 threads (s)	% of n.t.
1.0×10^{-2}	3.51	61	1.87	45	1.01	30	0.69	23
1.0×10^{-4}	3.81	52	2.01	36	1.07	23	0.72	17
1.0×10^{-6}	4.96	43	2.60	28	1.37	17	0.94	12.6
1.0×10^{-8}	4.68	23	2.45	13	1.29	7.5	0.87	5.2
1.0×10^{-10}	2.59	5.2	1.33	2.8	0.96	1.4	0.43	0.90

Notes: Results are for ETD35 applied to the Burgers test case. Thread times correspond to the costs associated with evaluating ETD functions using the specified number of threads. The quantity n.t. is the net time and includes nonlinear response, RK stage update, and ETD function costs.

all. For comparison, adapting costs for ETD35 on Burger's and NLS are shown in Table 4. In each test simulation, a 32-point contour of radius one was used to evaluate ETD functions for modes with $hc < 0.01$.

Parallelization can significantly reduce ETD function evaluation costs. This is demonstrated in Table 5 where we have run ETD35 simulations on the Burgers test case with 1, 2, 4, and 8 threads. It is seen in the table that simulations with bigger relative error tolerance levels benefited the most since a large percentage of their computation time is attributed to ETD evaluations. While the speed of ETD35 seen in Fig. 4(c) can be improved using parallelization, IF34 and IFDP remain better overall performers for the Burgers test case. We also note that the percentage of total time evaluating ETD functions shown in Table 5 is computed using serial code in the RK stages and inside the nonlinear response. Parallelizing the entire code could impact the relative distribution of costs but we leave that for future investigation.

5.2. Error control

There are some flaws with embedded error control for RKIF and RKETD. Ideally, the relative error computed will always be less than the specified error tolerance ϵ_{rel} . We rarely found this to be true as can be seen in Fig. 5. Often the relative error was significantly larger than ϵ_{rel} , particularly for the IF34 and ETD34 methods. The ETD35 and IFDP45 were somewhat better at satisfying the specified error tolerances but neither could be relied on with any confidence. Normally this would not be a major issue since we could simply adjust the safety factor ν lower. However, the effectiveness of tolerance levels depended to a large extent on what PDE was tested; error controls for the KS and Burgers models tended to be closer to target levels than the KdV and NLS models. The parameter ν is inherently part of the method, and hence it should not depend on what model is propagated.

A second important feature we look for is a linear relationship between tolerance levels and the resulting relative error; if we decrease the tolerance level by an order of magnitude, we hope the error will decrease by an order of magnitude. While the IF34 and ETD34 exhibit this desirable behavior for Burgers' model in Fig. 5(c) and the NLS model in Fig. 5(d), we do not see it as frequently or consistently as one would hope. In many cases, we simply see a general trend of relative error decreasing as ϵ_{rel} decreases. There are several plausible causes for a break down in error control. Local extrapolation can inherently cause suboptimal step adapting since the error control is based on the lower order embedded RK rather than the one being propagated. Furthermore, even among adaptive step strategies with local extrapolation, we choose simplicity over performance and it is probable that better error control is achievable [24]. Another issue for error control of ETD and IF methods is order reduction. For certain ranges of step sizes, the two embedded RK methods may exhibit the same order and the error estimate used in step adapting will be corrupted. While we have not seen any catastrophic loss of error control due to order reduction, we do believe it is negatively impacting efficiency. It should also be pointed out that order enhancement can occur such that convergence occurs at a rate faster than expected. If this occurs in the lower order embedded RK formula, then step size control will be negatively impacted.

One error control issue which was only experienced by ETD methods was a saturation in the effectiveness of ϵ_{rel} . There is a point at which further reductions of ϵ_{rel} have little or no impact in reducing the relative error. This is observed in Fig. 5 for both ETD34 and ETD35 in the KdV case and for ETD35 in Burgers' example. The saturation can be reduced or eliminated by increasing the linear mode cutoff point $hc < 0.01$ for which a contour integral is used to evaluate the ETD functions. While increasing the cutoff helps with saturation it also adds to the time spent evaluating ETD functions and hence the tradeoff needs to be considered carefully.

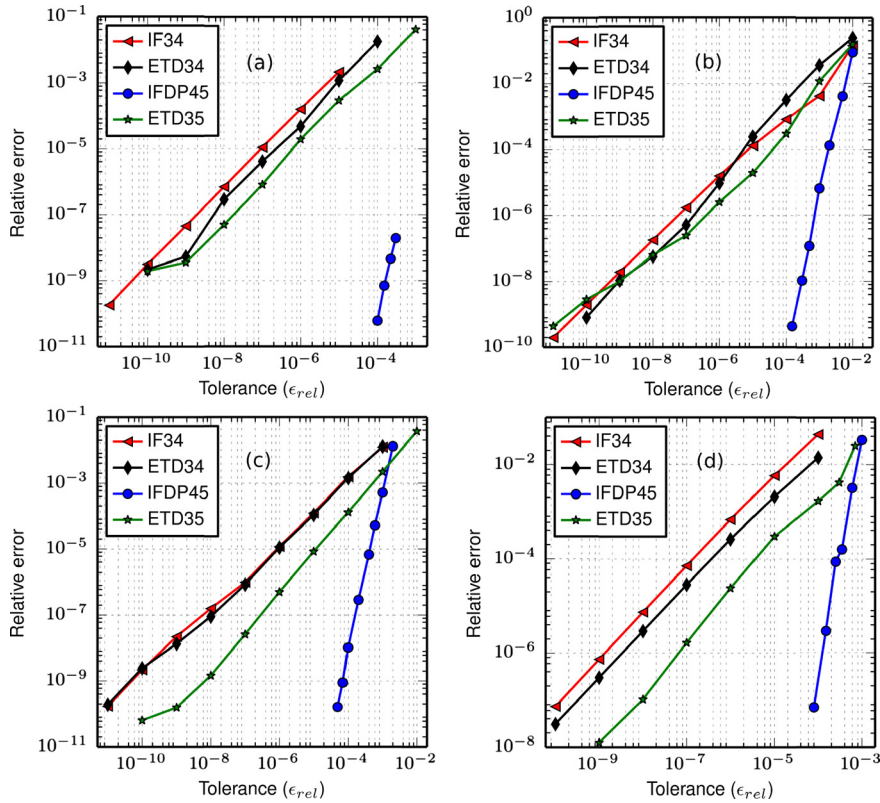


Fig. 5. Error control of several embedded RK methods for stiff systems on (a) KdV solitons, (b) KS chaos, (c) Burgers' shock fronts, and (d) NLS critical collapse with plasma defocusing and nonlinear absorption regularization mechanisms.

5.3. Adaptive step vs. constant step

One remaining question we have yet to answer is how the RKETD algorithms compare to their constant step higher order RK embeddings. In Fig. 6, we compare the computational efficiency of ETD34 to Krogstad's fourth order constant step method and ETD35 to its fifth order embedding (ETD5) for each of the test PDE's. In both the constant step and adaptive step cases, the ETD functions are computed using a 32-point contour integral for small linear modes $hc < 0.01$ and exactly otherwise. The constant step ETD functions are evaluated only once before propagation starts whereas the adaptive step ETD functions are evaluated whenever the step size adjusts.

For the NLS example, adaptive stepping was about 10 times faster than the equivalent constant step ETD. The gains in speed for the Burgers test case were not as dramatic and depended to a large extent on the specified tolerance level. For example, ETD35 was actually slower than ETD5 for mild ϵ_{rel} but 3–4 times faster for very stringent tolerances. This effect is due to ETD function costs which can be significantly reduced by parallelization as was shown in Table 5. Alternatively, one could adjust μ_c and μ_e such that the step size changes less frequently. This will typically result in smaller step sizes and more RK stage and nonlinear function evaluations, therefore the better approach is parallelization when feasible.

The KdV example had about the same efficiency for both the adaptive stepping ETD and constant stepping ETD. After looking at the evolution of solitons in Fig. 1(a), which had minimal step adapting, this result is to be expected. The KS example came off the worst of all, with the constant step ETD typically running faster. We believe this has to do with the chaotic behavior of the PDE which makes it hard for the numerical propagation to settle into a predictable pattern of step increases and decreases. Instead, the step increases and decreases tend to oscillate, each time adding to the ETD function costs, and averaging out to relatively small changes in the step size.

There is also an advantage to using embedded RKETD over constant stepping that has nothing to do with speed. When simulating a PDE, we are generally interested in the accuracy of the numerical solution which can be judged by relative error. The connection between relative error and relative error tolerance is more straightforward than the connection between relative error and step size. Even though error control is not great for ETD34 and ETD35, as was shown in Section 5.2, tolerance levels still give rough estimates for the amount of error generated during numerical simulations. To know what kind of relative error a constant step method produces requires a convergence study which may not be desirable or feasible.

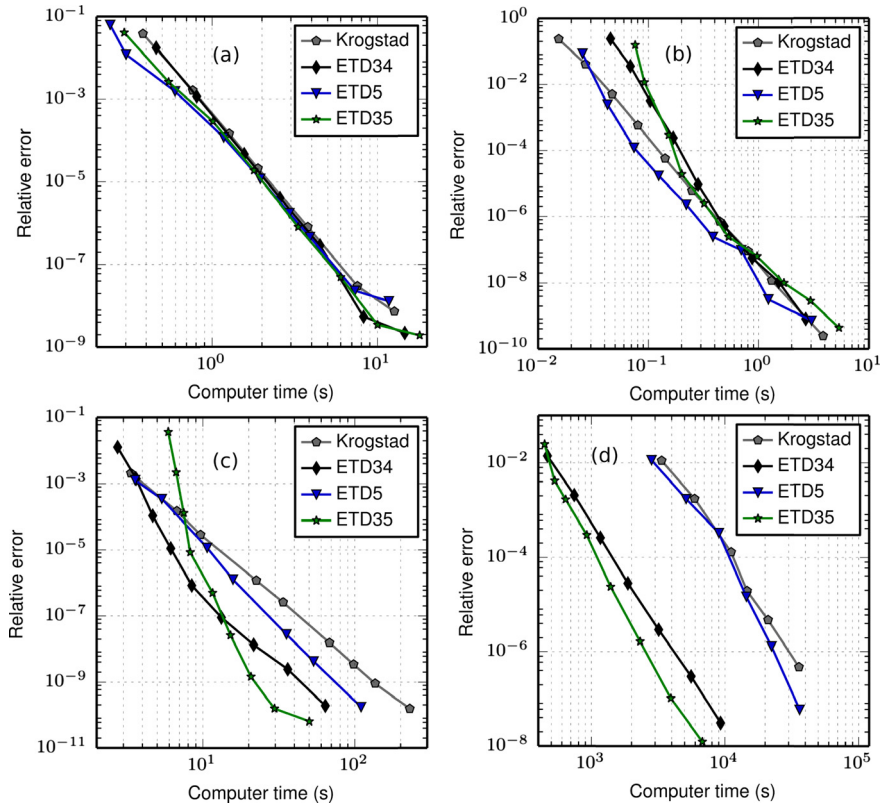


Fig. 6. Computational efficiency of adaptive stepping vs. constant stepping for NLS critical collapse with plasma defocusing and nonlinear absorption regularization mechanisms.

5.4. A non-diagonal test case

To show that our embedded methods are not likely to extend well to non-diagonal systems, we tested the Allen–Cahn PDE problem

$$u_t = \epsilon u_{xx} + u - u^3, \quad x \in [-1, 1],$$

$$u(x, t = 0) = 0.53x + 0.47 \sin(-1.5\pi x),$$

with boundary conditions $u(-1, t) = -1$, and $u(1, t) = 1$ (see [5]). The spatial variable is discretized on a Chebyshev grid and a change of variable to $w = u - x$ is performed such that $w(-1, t) = w(1, t) = 0$. For the test case, we took $\epsilon = 0.001$ and propagated to $t = 3$. The operator $\mathcal{L} = \epsilon \partial_x^2$ is discretized to $L = \epsilon D_2$ where D_2 is a full non-diagonal second order differentiation matrix. The first and last rows of L are set to zero to enforce the boundary conditions. Matrix exponential functions $\exp(hc_i L)$ for the RK stages $i = 1, \dots, s$, are computed using a Padé scaling and squaring algorithm [30,31]. The (q, j) Padé functions are chosen such that $q = 6$ and j is determined by the norm of the argument of the matrix exponential. For the ETD functions $a_{ij}(hL)$ and $b_i(hL)$, we use the contour evaluation technique [5]

$$f(hL) = \frac{1}{2\pi i} \int_{\Gamma} f(t)(tI - hL)^{-1} dt,$$

where we select our contour Γ to be a circle of radius $r = 20$. Since L is real, we integrate over the upper-half of the complex plane and take the real part of the result. The contour path is discretized with the points $t_k = r \exp(i(k - 0.5)\pi/M)$ for $k = 1, 2, \dots, M$, such that the contour integral is approximated by

$$f(hL) = \frac{1}{M} \operatorname{Re} \left\{ \sum_{k=1}^M t_k f(t_k) (t_k I - hL)^{-1} \right\}. \quad (13)$$

With $N = 80$ spatial points and $M = 36$ contour points, the Allen–Cahn test problem is propagated for various step sizes using Krogstad's fourth order constant step method [6]. Our results are shown in Table 6 where the CPU times are based on a C++ implementation of Krogstad's method using the Eigen software package [32]. The ETD contour inverses are computed

Table 6

Krogstad step update and ETD function evaluation costs for a non-diagonal Allen–Cahn test PDE.

h	Step (s)	ETD (s)	$(tI - hL)^{-1}$ (s)	$\exp(hL)$ (s)	Relative error
0.75	1.88×10^{-4}	1.43×10^{-1}	6.90×10^{-2}	4.51×10^{-3}	3.33×10^{-3}
0.5	2.55×10^{-4}	1.41×10^{-1}	6.82×10^{-2}	4.07×10^{-3}	4.70×10^{-4}
0.2	6.45×10^{-4}	1.41×10^{-1}	6.82×10^{-2}	4.03×10^{-3}	8.75×10^{-6}
0.1	1.26×10^{-3}	1.41×10^{-1}	6.86×10^{-2}	3.74×10^{-3}	4.94×10^{-7}
0.01	1.23×10^{-2}	1.39×10^{-1}	6.83×10^{-2}	3.25×10^{-3}	2.57×10^{-10}

Notes: The $(tI - hL)^{-1}$ CPU time includes the time associated with inverting each $(t_k I - hL)^{-1}$ for $k = 1, \dots, M$ in the evaluation of the ETD contours (see Eq. (13)). For this particular case we use $M = 36$ and note that fewer points will start to impact the accuracy of the solution.

using the Eigen LU algorithm with full pivoting, and the scaling and squaring algorithm employs a Householder QR algorithm with full pivoting. The constant step case is examined since it requires the minimal amount of ETD function computations. The results, shown in Table 6, indicate that the cost of evaluating the ETD functions is larger than the cost of updating the RK stages for each of the step sizes tested. This is despite the fact that the RK stages are updated after each step while the ETD coefficients are only evaluated at the startup of the propagation.

We have also shown two major sources of the ETD function CPU time, namely the time spent evaluating $\exp(hL)$, and the time spent evaluating matrix inversions $(t_k I - hL)^{-1}$ for $k = 1, 2, \dots, M$. As was true for diagonal operators, the contour approach to evaluating ETD functions can be parallelized with relative efficiency. Other gains in CPU speed may be achievable, for instance by using alternative algorithms for computing the matrix exponentials $\exp(c_i hL)$ [33] and ETD functions $\psi_j(c_i hL)$ [34,14]. However, each algorithm for computing ETD functions will require $O(N^3)$ flops for a dense matrix which compares to the $O(N^2)$ cost of updating the RK stages.

We have not tested a finite difference approximation to the operator $\mathcal{L} = \epsilon \partial_x^2$ but expect embedded RKETD methods will still be inefficient in this circumstance. Because the finite difference scheme produces a banded matrix L , the a_{ij} and b_i computations can be reduced to $O(N^2)$ flops. However, the factor in front of the $O(N^2)$ may be large since the inverses in Eq. (13) are dense even if L is banded. The scaling and squaring algorithm used for computing the $\exp(c_i hL)$ functions benefits even less from the bandedness of L than the contour algorithm; the squaring part of the algorithm multiplies dense matrices together and hence still costs $O(N^3)$ flops. In general, alternative algorithms should be considered for adaptive stepping of non-diagonal systems.

The best way to handle adaptive stepping for non-diagonal operators appears to be with exponential integrators that use Krylov subspace methods [15,16]. These methods are similar to ETD in nature though there are several important distinctions. The Krylov exponential integrators compute approximations to the matrix vector products $\exp(c_i hL)u_n$, $a_{ij}(c_i hL)k_j$, and $b_i(hL)k_i$, in a Krylov subspace rather than directly computing ETD functions. The advantage of this, especially in the case of large non-diagonal systems, is that the Krylov subspace dimension can be significantly smaller than the dimension of L and hence the matrix vector ETD approximations can be computed relatively quickly. A discussion of the design of efficient Krylov exponential integrators is presented in [35]. However, for small non-diagonal systems we expect that adaptive stepping using Krylov methods is inefficient for many of the same reasons we expect it to be inefficient for embedded RKETD methods. In particular, when the dimension of the Krylov subspace is not much smaller than the dimension of L , then the costs associated with evaluating the ETD matrix vector products at each time step will make the algorithm inefficient.

In certain cases, one might consider propagating the diagonalized embedded RKETD system

$$\begin{aligned}
 v_n &= S^{-1}u_n, \\
 v_{n+1} &= e^{h\Lambda}v_n + h \sum_{i=1}^s b_i(h\Lambda)S^{-1}\mathcal{N}(Sm_i), \\
 m_i &= e^{hc_i\Lambda}v_n + h \sum_{j=1}^{i-1} a_{ij}(h\Lambda)S^{-1}\mathcal{N}(Sm_j), \quad i = 1, 2, \dots, s \\
 u_{n+1} &= Sv_{n+1},
 \end{aligned} \tag{14}$$

where $L = SAS^{-1}$. This will shift the burden of costs away from the ETD functions to the RK stages which could make adaptive stepping feasible. However, this approach will not work in general and should be avoided when the eigenvector matrix S is ill-conditioned [31]. For the Allen–Cahn test problem, $\text{cond}(S) = 15.0$ and stability was not an issue. Finite difference approximations to $\mathcal{L} = \epsilon \partial_x^2$ would generally be ideal since a symmetric L produces an orthogonal eigenvector matrix. However, when boundary conditions are enforced through L , this can corrupt the symmetry; for example, a second order finite difference approximation to $\mathcal{L} = \epsilon \partial_x^2$ with the first and last rows of L zeroed out to enforce Dirichlet boundary conditions, yields $\text{cond}(S) = 197.9$. Therefore, it is advisable to handle the boundary conditions separately when using a diagonalized RKETD method; that is, it would be better to remove the first and last rows and columns in L in order to solve for just the interior spatial points with the boundary conditions explicitly enforced.

The diagonalized version of Krogstad's fourth order ETD method is applied to the Allen–Cahn test problem with computational results shown in Table 7. The eigenvalues and eigenvectors of L are determined using a general purpose

Table 7

Diagonalized Krogstad step update and ETD function evaluation costs for an Allen–Cahn test PDE.

h	Step (s)	ETD (s)	SAS^{-1} (s)	Relative error
0.75	3.60×10^{-4}	3.72×10^{-4}	7.33×10^{-3}	3.33×10^{-3}
0.5	5.37×10^{-4}	3.51×10^{-4}	7.34×10^{-3}	4.70×10^{-4}
0.2	1.32×10^{-3}	4.63×10^{-4}	7.19×10^{-3}	8.75×10^{-6}
0.1	2.62×10^{-3}	5.75×10^{-4}	7.23×10^{-3}	4.94×10^{-7}
0.01	2.59×10^{-2}	1.39×10^{-3}	7.05×10^{-3}	4.91×10^{-11}

Notes: The SAS^{-1} CPU time includes the time associated with finding the eigenvalues and eigenvectors of L and inverting S , the resulting eigenvector matrix. For the ETD functions, an $M = 36$ point contour integration was performed for eigenvalues with $\lambda h < 0.01$.

decomposition algorithm from the Eigen software package [32]. It should be emphasized that the diagonalization procedure occurs only once, at the start of propagation, and changing the step size does not require eigenvalues and eigenvectors to be recomputed. Therefore, updates to the ETD functions can be done in $O(N)$ flops rather than the $O(N^3)$ flops needed in the standard RKETD formulation. For non-diagonal L , it is clear that the cheaper ETD updates are needed for the embedded RKETD algorithms to efficiently employ an adaptive step.

A major advantage for Krylov exponential integrators over diagonalized ETD is that they will work for general L . Therefore, they should be the preferred method for adaptive stepping of a non-diagonal system. However, for small systems and specialized L , there may be efficiency gains available by using a diagonalized adaptive step RKETD and hence a more complete study of these methods is warranted.

6. Conclusion

We have investigated explicit embedded Runge–Kutta adaptive stepping for stiff or highly oscillatory nonlinear systems. Several well known stiff constant step RK algorithms, including Krogstad's ETD RK4-B and integrating factor based on standard fourth order RK, were modified to include an embedded error estimate. Using stiff order conditions and a symbolic math package (Mathematica), we constructed a completely new embedded Runge–Kutta exponential time-differencing method we call ETD35. Lastly, we converted a common embedded explicit RK algorithm into an integrating factor method, namely Dormand Prince's RK45.

An adaptive step strategy, differing only slightly from continuous RK theory, was presented. Error control based on this adaptive step algorithm provided efficiency gains over constant step methods in certain cases but also had flaws. Improved embedded error control for RKETD is necessary for the methods to become broadly applicable.

In tests on several diagonalizable nonlinear PDE's, the best method tended to depend on how costly ETD function updates were versus the cost of nonlinear function evaluations. When ETD function costs were relatively cheap, the ETD methods were typically faster but in situations where ETD function updates were costly, the IF methods were faster. For non-diagonal systems, we showed that ETD function evaluation costs can make adaptive stepping inefficient for embedded RKETD and discussed alternative algorithms employing Krylov subspace projections. We also suggested further study of diagonalized embedded RKETD algorithms but cautioned that they are not as robust as Krylov methods.

A new way of computing stability regions for ETD methods was proposed. This procedure applies easily to both diagonal and non-diagonal linear operators and is demonstrated for Krogstad's ETD RK4-B in a short MATLAB script included in Appendix A.

Implementation of embedded RKETD takes about as much effort as for non-stiff embedded RK. Modest additional effort is needed in coding algorithms related to ETD function computations. For this, the contour integral approach is particularly easy to understand and implement while the scaling and squaring approach is more complicated but also more generally applicable. Once an embedded RKETD solver has been coded, it should accept generic operators $\mathcal{L}$ and $\mathcal{N}$ which are supplied by the user of the algorithm in much the same way as a typical RK adaptive step solver takes in a right-hand-side function. Therefore, almost all of the work is done upfront in constructing the solver and relatively little has to be done by the user.

Acknowledgements

This work was supported by AFOSR MURI, "Mathematical modeling and experimental validation of ultrashort nonlinear light-matter coupling associated with filamentation in transparent media," Grant FA9550-10-1-0561. M. Brio was also supported in part by US AFOSR BRI Grant FA9550-12-1-0482.

Appendix A. MATLAB ETD stability script

```
% StableETD34.m - stability regions for ETD34
```

```
% Stability regions depend on the linear operator L (hL)
% L is specified as a matrix and can be diagonal or non-diagonal
% Contour integration added for small hL eigenvalues,
```

```

% See Kassam and Trefethen, Siam J. Sci. Comput. Vol. 26, 2005

% Specify linear operator
N = 20;
%L = cheb(N+1); L = L^2; L = 0.0025*L(2:N+1,2:N+1);
L = diag(linspace(-2i*pi,2i*pi,N));
%L = diag(linspace(-8,-0.01,N));

% Nonlinear grid specifications (linearized nonlinear term)
M = 100;
x = linspace(-5,5,M);
y = linspace(-5,5,M);
[X,Y] = meshgrid(x,y);
W = X+1i*Y;

% Evaluate ETD functions
R = 64;
r = 15*exp(2i*pi*((1:R)-0.5)/R);
I = eye(N); Z = zeros(N);
a21 = Z; a31 = Z; a32 = Z; a41 = Z; a43 = Z; a51 = Z; a52 = Z; a53 = Z;
a54 = Z; a61 = Z; a62 = Z; a63 = Z; a65 = Z;

R = 64;
r = 15*exp(2i*pi*((1:R)-0.5)/R);
I = eye(N); Z = zeros(N);
a21 = Z; a31 = Z; a32 = Z; a41 = Z; a43 = Z; a51 = Z; a52 = Z; a53 = Z;
a54 = Z; a61 = Z; a62 = Z; a63 = Z; a65 = Z;
for j = 1:R
    z = r(j);
    zIA = inv(z*I - L);
    a21 = a21 + zIA*(exp(z/2)-1);
    a31 = a31 + zIA*(4+exp(z/2)*(-4+z)+z)/z;
    a32 = a32 - 2*zIA*(2-2*exp(z/2)+z)/z;
    a41 = a41 + zIA*(2+exp(z)*(-2+z)+z)/z;
    a43 = a43 - 2*zIA*(1-exp(z)+z)/z;
    a51 = a51 + zIA*(-4-z+exp(z)*(4-3*z+z^2))/z^2;
    a52 = a52 + 2*zIA*(2+exp(z)*(-2+z)+z)/z^2;
    a53 = a53 + 2*zIA*(2+exp(z)*(-2+z)+z)/z^2;
    a54 = a54 - zIA*(4+exp(z)*(-4+z)+3*z+z^2)/z^2;
    a61 = a61 + zIA*(-4-z+exp(z)*(4-3*z+z^2))/z^2;
    a62 = a62 + 2*zIA*(2+exp(z)*(-2+z)+z)/z^2;
    a63 = a63 + 2*zIA*(2+exp(z)*(-2+z)+z)/z^2;
    a65 = a65 - zIA*(4+exp(z)*(-4+z)+3*z+z^2)/z^2;
end

a21 = a21/R; a31 = a31/R; a32 = a32/R; a41 = a41/R; a43 = a43/R;
a51 = a51/R; a52 = a52/R; a53 = a53/R; a54 = a54/R;
a61 = a61/R; a62 = a62/R; a63 = a63/R; a65 = a65/R;

% Compute stability regions
E2 = expm(L/2); E = expm(L);
G = zeros(M); H = G;
k2 = zeros(N); k3 = k2; k4 = k2; k5 = k2; k6 = k2;
for m = 1:M
    for n = 1:M
        k2 = E2+W(m,n)*a21;
        k3 = E2+W(m,n)*(a31 + a32*k2);
        k4 = E + W(m,n)*(a41 + a43*k3);
        k5 = E + W(m,n)*(a51 + a52*k2 + a53*k3 + a54*k4);
        k6 = E + W(m,n)*(a61 + a62*k2 + a63*k3 + a65*k5);
    % Compute spectral radius

```

```

    G(m,n) = norm(k5);
    H(m,n) = norm(k6);
end
end

figure(1), contour(X,Y,G,[1 1],'k'); title('Krogstad ETDK4-B');
figure(2), contour(X,Y,H,[1 1],'k'); title('Embedded 3rd Order');

```

Appendix B. Mathematica ETD5 order conditions

The Mathematica script used for satisfying our fifth order ETD method's order conditions is provided in this appendix. With some modifications it can be extended to include embedded methods, other fifth order methods with different stability regions, or ETD methods of lower or higher order. The particular script included solves for a family of fifth order, stiff order three, ETD methods with the free parameters a421, a422, a522, a532, a622, a632, b33, and b43. These free parameters must be selected with some care, otherwise the resulting stability regions may be undesirable or the method may have poor convergence rates. We opted to focus on the stability region of our fifth order method when selecting the free parameters since our primary concern was achieving stability for as large a class of PDE problems as possible. Alternatively, one may wish to optimize an ETD method for speed on a specific PDE problem; we have not attempted this type of optimization.

With respect to optimizing stability, we share our process. The MATLAB code in [Appendix A](#) can be modified to plot the stability contours of the family of fifth order methods generated by the Mathematica script from this section. Furthermore, we can write the modified stability code as a function, with the free parameters as the input and an output corresponding to the area of a contour region. The specific contour region of interest is dependent on the linear operator $\mathcal{L}$, which for our purposes we took to be $\mathcal{L} \in [-2\pi i, 2\pi i]$. We also wanted to maximize the stability area near the imaginary axis and hence only accounted for this area in the function output. This is done such that PDE problems like KdV will be stable with our resulting fifth order ETD method. The function we create is then fed into the MATLAB genetic algorithm (G.A.) optimization toolbox. We found that this yielded a large number of fifth order ETD methods with acceptable stability regions. Since the optimization was overdetermined, we were able to choose some free parameters based on other criteria such as convergence rates, stability regions for say $\mathcal{L} \in [-8, -0.01]$, and reducing the number of ETD function evaluations in our method. Setting these extra free parameters to zero tended to yield methods with relatively good convergence rates and hence that's how we selected a421 = a422 = a522 = a532 = a622 = 0. The G.A. was then reapplied with just the free parameters a632, b33, and b43. Even with the reduced number of free parameters, the system is still overdetermined and hence several sets of a632, b33, and b43 yield stability regions with about the same area. With these final parameter sets, we checked the corresponding convergence rates and stability regions before settling on the ones used in this paper and seen in the Mathematica script.

Mathematica Construction of RKETD5 Method and Order Verification
P. Whalen, May 2014

```

<< NumericalDifferentialEquationAnalysis`
Unprotect[Power];
Power[0, 0] = 1;
Protect[Power];

```

Construct RKETD Functions

```

Clear["Global`*"]
D1 = Dt[NL[u], {t, 1}]; D1 = D1 /. Dt[u, t] -> (Lu + NL[u]);
D2 = Dt[D1, t]; D2 = Simplify[D2 /. {Dt[L, t] -> 0, Dt[u, t] -> Lu + NL[u]}];
D3 = Dt[D2, t]; D3 = Simplify[D3 /. {Dt[u, t] -> Lu + NL[u], Dt[L, t] -> 0}];
D4 = Dt[D3, t]; D4 = Simplify[D4 /. {Dt[u, t] -> Lu + NL[u], Dt[L, t] -> 0}];
D5 = Dt[D4, t]; D5 = Simplify[D5 /. {Dt[u, t] -> Lu + NL[u], Dt[L, t] -> 0}];
D6 = Dt[D5, t]; D6 = D6 /. {Dt[u, t] -> Lu + NL[u], Dt[L, t] -> 0};
D7 = Dt[D6, t]; D7 = D7 /. {Dt[u, t] -> Lu + NL[u], Dt[L, t] -> 0};
DMat = {NL[u], D1, D2, D3, D4, D5, D6, D7};

ψf[k_, z_] := k! 
$$\frac{\left(\text{Exp}[z] - \text{Sum}\left[\frac{z^p}{p!}, \{p, 0, k-1\}\right]\right)}{z^k};$$

p = 5; s = 6; stiffint = {0, 1, 2, 2, 2, 2, 3};

```

```

c = Table[ToExpression["c" <> ToString[i]], {i, 1, s+1}];
c[[1]] = 0; c[[s+1]] = 1;

Base = Table[ $\frac{(c[[i]]^j)}{j}$  Subscript[ $\psi$ , j], {i, 1, s+1}, {j, 1, stiffint[[i]]}];

A = Table[ToExpression["a" <> ToString[i] <> ToString[j]], {i, 1, s}, {j, 1, i-1}];
b = Table[ToExpression["b" <> ToString[i]], {i, 1, s}];

c2 =  $\frac{1}{4}$ ; c3 =  $\frac{1}{4}$ ; c4 =  $\frac{1}{2}$ ; c5 =  $\frac{3}{4}$ ; c6 = 1; c7 = 1;

Q = Table[c[[j]]^i, {i, 0, stiffint[[2]]-1}, {j, 1, 1}];
x = Q.A[[2]];
sol = Solve[x[[1]] == Base[[2, 1]], A[[2]]]
A[[2]] = A[[2]] /. sol[[1]];

Q = Table[c[[j]]^i, {i, 0, stiffint[[3]]-1}, {j, 1, 2}];
x = Q.A[[3]];
sol = Solve[x[[1]] == Base[[3, 1]] && x[[2]] == Base[[3, 2]], A[[3]]]
A[[3]] = A[[3]] /. sol[[1]];

Q = Table[c[[j]]^i, {i, 0, stiffint[[4]]-1}, {j, 1, 3}];
x = Q.A[[4]];
sol = Solve[x[[1]] == Base[[4, 1]] && x[[2]] == Base[[4, 2]], {a41, a43}]
A[[4]] = A[[4]] /. sol[[1]];

Q = Table[c[[j]]^i, {i, 0, stiffint[[5]]-1}, {j, 1, 4}];
x = Q.A[[5]];
sol = Solve[x[[1]] == Base[[5, 1]] && x[[2]] == Base[[5, 2]], {a51, a54}]
A[[5]] = A[[5]] /. sol[[1]];

Q = Table[c[[j]]^i, {i, 0, stiffint[[6]]-1}, {j, 1, 5}];
x = Q.A[[6]];
sol = Solve[x[[1]] == Base[[6, 1]] && x[[2]] == Base[[6, 2]], {a61, a65}]
A[[6]] = A[[6]] /. sol[[1]];

Q = Table[c[[j]]^i, {i, 0, stiffint[[7]]-1}, {j, 1, 6}];
x = Q.b;
sol = Solve[
  x[[1]] == Base[[7, 1]] && x[[2]] == Base[[7, 2]] && x[[3]] == Base[[7, 3]], {b1, b2, b6}]
b = b /. sol[[1]];

 $\left\{ \left\{ a_{21} \rightarrow \frac{\psi_1}{4} \right\} \right\}$ 

 $\left\{ \left\{ a_{31} \rightarrow \frac{1}{8} (2 \psi_1 - \psi_2), a_{32} \rightarrow \frac{\psi_2}{8} \right\} \right\}$ 

 $\left\{ \left\{ a_{41} \rightarrow \frac{1}{2} (\psi_1 - \psi_2), a_{43} \rightarrow -a_{42} + \frac{\psi_2}{2} \right\} \right\}$ 

 $\left\{ \left\{ a_{51} \rightarrow \frac{1}{16} (-8 a_{52} - 8 a_{53} + 12 \psi_1 - 9 \psi_2), a_{54} \rightarrow \frac{1}{16} (-8 a_{52} - 8 a_{53} + 9 \psi_2) \right\} \right\}$ 

 $\left\{ \left\{ a_{61} \rightarrow \frac{1}{3} (-2 a_{62} - 2 a_{63} - a_{64} + 3 \psi_1 - 2 \psi_2), a_{65} \rightarrow \frac{1}{3} (-a_{62} - a_{63} - 2 a_{64} + 2 \psi_2) \right\} \right\}$ 

 $\left\{ \left\{ b_1 \rightarrow \frac{1}{6} (3 b_4 + 3 b_5 + 6 \psi_1 - 15 \psi_2 + 8 \psi_3), \right. \right.$ 

```

$$b_2 \rightarrow \frac{1}{9} (-9 b_3 - 12 b_4 - 9 b_5 + 24 \psi_2 - 16 \psi_3), b_6 \rightarrow -\frac{b_4}{6} - \frac{b_5}{2} - \frac{\psi_2}{6} + \frac{4 \psi_3}{9} \}} \}$$

(*Free Parameters*)

```
a42 = a421  $\psi_1$  + a422  $\psi_2$ ; a52 = a521  $\psi_1$  + a522  $\psi_2$ ; a53 = a531  $\psi_1$  + a532  $\psi_2$ ;
a62 = a621  $\psi_1$  + a622  $\psi_2$ ; a63 = a631  $\psi_1$  + a632  $\psi_2$ ; a64 = a641  $\psi_1$  + a642  $\psi_2$ ;
b3 = b31  $\psi_1$  + b32  $\psi_2$  + b33  $\psi_3$ ; b4 = b41  $\psi_1$  + b42  $\psi_2$  + b43  $\psi_3$ ;
b5 = b51  $\psi_1$  + b52  $\psi_2$  + b53  $\psi_3$ ;
```

(*Underlying RK fifth order conditions*)

```
Clear[i, j]
AU = Table[Sum[Coefficient[A[[i, j]], Subscript[ $\psi$ , k]], {k, 1, stiffint[[i]]}],
  {i, 1, s}, {j, 1, i-1}];
bU = Table[Sum[Coefficient[b[[i]], Subscript[ $\psi$ , k]], {k, 1, stiffint[[s+1]]}], {i, 1, s}];
RK = RungeKuttaOrderConditions[p, s, RungeKuttaMethod → Explicit];
RK = RK /.  $\dot{c}_1 \rightarrow 0$ ;
RK = RK /. Subscript[ $\dot{c}$ , i_] → c[[i]];
RK = RK /. Subscript[ $\dot{a}$ , i_, j_] → AU[[i, j]];
RK = RK /. Subscript[ $\dot{b}$ , i_] → bU[[i]];
Map[Length, RK];

paramList = {a521, a531, a621, a631, a641, b31, b41, b51};
sol = Solve[RK[[3, 1]] && RK[[3, 2]] && RK[[4, 1]] && RK[[4, 2]] &&
  RK[[4, 3]] && RK[[4, 4]] && RK[[5, 1]] && RK[[5, 2]] && RK[[5, 3]] && RK[[5, 4]] &&
  RK[[5, 5]] && RK[[5, 6]] && RK[[5, 7]] && RK[[5, 8]] && RK[[5, 9]], paramList]

{ {a521 →  $\frac{1}{8} (-3 - 6 a421 - 6 a422 - 8 a522)$ , a531 →  $\frac{1}{8} (3 + 6 a421 + 6 a422 - 8 a532)$ ,
  a621 →  $\frac{1}{7} (8 + 12 a421 + 12 a422 - 7 a622)$ , a631 →  $\frac{1}{7} (6 - 12 a421 - 12 a422 - 7 a632)$ ,
  a641 →  $\frac{1}{7} (-12 - 7 a642)$ , b31 →  $\frac{1}{45} (16 - 45 b32 - 45 b33)$ ,
  b41 →  $\frac{1}{15} (2 - 15 b42 - 15 b43)$ , b51 →  $\frac{1}{45} (16 - 45 b52 - 45 b53)$  } }
```

```
pList = paramList /. sol[[1]];
a521 = pList[[1]]; a531 = pList[[2]]; a621 = pList[[3]]; a631 = pList[[4]];
a641 = pList[[5]]; b31 = pList[[6]]; b41 = pList[[7]]; b51 = pList[[8]];
```

(*RKETD method error*)

```
As = Table[Sum[Coefficient[A[[i, j]], Subscript[ $\psi$ , k]]  $\psi f[k, c[[i]] hL]$ ,
  {k, 1, stiffint[[i]]}], {i, 1, s}, {j, 1, i-1}];
bs = Table[Sum[Coefficient[b[[i]], Subscript[ $\psi$ , k]]  $\psi f[k, hL]$ ,
  {k, 1, stiffint[[s+1]]}], {i, 1, s}];
p = 5;
k1 = u;
For[i = 2, i <= s, i++, Subscript[k, i] =
  Exp[c[[i]] hL] u + h Sum[Series[As[[i, j]], {h, 0, p}] NL[Subscript[k, j]], {j, 1, i-1}]]
rktay = Sum[Series[h (bs[[i]] NL[Series[Subscript[k, i], {h, 0, p}]]], {h, 0, p+1}],
  {i, 1, s}];
tay = Series[Sum[ $\frac{h^i}{i!} \psi f[i, hL]$  DMat[[i]], {i, 1, p+1}], {h, 0, p+1}];
err = Simplify[rktay - tay];
```

```
err = err /. { NL[u] → NL,
  Derivative[1][NL][u] → Derivative[1][NL], Derivative[2][NL][u] → Derivative[2][NL],
  Derivative[3][NL][u] → Derivative[3][NL], Derivative[4][NL][u] → Derivative[4][NL],
  Derivative[5][NL][u] → Derivative[5][NL], Derivative[6][NL][u] → Derivative[6][NL]};
```

(*Remaining fifth order restrictions*)

```
paramList = {b32, b42, b52, b53, a642};
F4 = Coefficient[err, h4];
co41 = Coefficient[F4, (NL + L u) NL' L NL'];
F5 = Coefficient[err, h5];
co51 = Coefficient[F5, (NL + L u) L (NL')3];
co52 = Coefficient[F5, (NL + L u) NL' L (NL + L u) NL''];
co53 = Coefficient[F5, (NL + L u) (NL + L u) L2 NL''];
co54 = Coefficient[F5, (NL + L u) L2 (NL')2];
co55 = Coefficient[F5, (NL + L u) NL' L3];
sol =
  Solve[co41 == 0 && co51 == 0 && co52 == 0 && co53 == 0 && co54 == 0 && co55 == 0, paramList]
```

$$\left\{ \left\{ b32 \rightarrow -\frac{8}{15} - \frac{3 b33}{2} - b43 + \frac{(14 + 84 a422 + 336 a522 + 98 a622 + 675 b43 + 3375 a421 b43 + 3375 a422 b43)}{(675 (1 + 5 a421 + 5 a422))}, \right. \right.$$

$$b42 \rightarrow -\frac{(14 + 84 a422 + 336 a522 + 98 a622 + 675 b43 + 3375 a421 b43 + 3375 a422 b43)}{(450 (1 + 5 a421 + 5 a422))}, b52 \rightarrow \frac{1}{10} (32 + 15 b33 + 10 b43) +$$

$$\frac{(14 + 84 a422 + 336 a522 + 98 a622 + 675 b43 + 3375 a421 b43 + 3375 a422 b43)}{(675 (1 + 5 a421 + 5 a422))}, b53 \rightarrow -\frac{16}{9} - b33 - \frac{4 b43}{3},$$

$$a642 \rightarrow -\frac{22}{7} - \frac{12 a522}{7} - \frac{12 a532}{7} - a622 - a632 - \frac{675 b43}{392} +$$

$$\left. \left. \frac{(14 + 84 a422 + 336 a522 + 98 a622 + 675 b43 + 3375 a421 b43 + 3375 a422 b43)}{(392 (1 + 5 a421 + 5 a422))} \right\} \right\}$$

```
pList = paramList /. sol[[1]];
b32 = pList[[1]]; b42 = pList[[2]];
b52 = pList[[3]]; b53 = pList[[4]]; a642 = pList[[5]];
```

(*Error terms*)

```
Print[Simplify[Coefficient[err, h3]]];
Print[Simplify[Coefficient[err, h4]]];
Print[Simplify[Coefficient[err, h5]]];
```

(*ETD Method(s)*)

```
a421 = 0; a422 = 0; a522 = 0; a532 = 0;
a622 = 0; a632 =  $-\frac{87}{28}$ ; b33 =  $-\frac{1}{9}$ ; b43 =  $-\frac{6}{5}$ ;
Print["a21 = ", A[[2]]];
Print["a31,a32 = ", A[[3]]];
Print["a41,a42,a43 = ", A[[4]]];
Print["a51,a52,a53,a54 = ", A[[5]]];
Print["a61,a62,a63,a64,a65 = ", Simplify[A[[6]]]];
```

```
Print["b1,b2,b3,b4,b5,b6 = ", Simplify[b]];
Print["c: ", c]
```

$$a_{21} = \left\{ \frac{\psi_1}{4} \right\}$$

$$a_{31}, a_{32} = \left\{ \frac{1}{8} (2\psi_1 - \psi_2), \frac{\psi_2}{8} \right\}$$

$$a_{41}, a_{42}, a_{43} = \left\{ \frac{1}{2} (\psi_1 - \psi_2), 0, \frac{\psi_2}{2} \right\}$$

$$a_{51}, a_{52}, a_{53}, a_{54} = \left\{ \frac{1}{16} (12\psi_1 - 9\psi_2), -\frac{3\psi_1}{8}, \frac{3\psi_1}{8}, \frac{9\psi_2}{16} \right\}$$

$$a_{61}, a_{62}, a_{63}, a_{64}, a_{65} = \left\{ \frac{1}{42} (-77\psi_1 + 59\psi_2), \frac{8\psi_1}{7}, \frac{3}{28} (37\psi_1 - 29\psi_2), -\frac{12\psi_1}{7}, \frac{1}{84} (-47\psi_1 + 143\psi_2) \right\}$$

$$b_1, b_2, b_3, b_4, b_5, b_6 = \left\{ \frac{7(257\psi_1 - 497\psi_2 + 270\psi_3)}{2700}, 0, \frac{1097\psi_1 - 467\psi_2 - 150\psi_3}{1350}, \right. \\ \left. -\frac{2}{225} (49\psi_1 - 199\psi_2 + 135\psi_3), \frac{-313\psi_1 + 883\psi_2 - 90\psi_3}{1350}, \frac{509\psi_1 - 2129\psi_2 + 1830\psi_3}{2700} \right\}$$

$$c: \left\{ 0, \frac{1}{4}, \frac{1}{4}, \frac{1}{2}, \frac{3}{4}, 1, 1 \right\}$$

```
Print["a21 = ", As[[2]]]
Print["a31,a32 = ", As[[3]]];
Print["a41,a42,a43 = ", As[[4]]];
Print["a51,a52,a53,a54 = ", As[[5]]];
Print["a61,a62,a63,a64,a65 = ", Simplify[As[[6]]]];
Print["b1,b2,b3,b4,b5,b6 = ", Simplify[bs]];
Print["c: ", c]
```

$$a_{21} = \left\{ \frac{-1 + e^{\frac{hL}{4}}}{hL} \right\}$$

$$a_{31}, a_{32} = \left\{ \frac{-1 + e^{\frac{hL}{4}}}{hL} - \frac{4 \left(-1 + e^{\frac{hL}{4}} - \frac{hL}{4} \right)}{h^2 L^2}, \frac{4 \left(-1 + e^{\frac{hL}{4}} - \frac{hL}{4} \right)}{h^2 L^2} \right\}$$

$$a_{41}, a_{42}, a_{43} = \left\{ \frac{-1 + e^{\frac{hL}{2}}}{hL} - \frac{4 \left(-1 + e^{\frac{hL}{2}} - \frac{hL}{2} \right)}{h^2 L^2}, 0, \frac{4 \left(-1 + e^{\frac{hL}{2}} - \frac{hL}{2} \right)}{h^2 L^2} \right\}$$

$$a_{51}, a_{52}, a_{53}, a_{54} = \left\{ \frac{-1 + e^{\frac{3hL}{4}}}{hL} - \frac{2 \left(-1 + e^{\frac{3hL}{4}} - \frac{3hL}{4} \right)}{h^2 L^2}, -\frac{-1 + e^{\frac{3hL}{4}}}{2hL}, \frac{-1 + e^{\frac{3hL}{4}}}{2hL}, \frac{2 \left(-1 + e^{\frac{3hL}{4}} - \frac{3hL}{4} \right)}{h^2 L^2} \right\}$$

$$a_{61}, a_{62}, a_{63}, a_{64}, a_{65} = \left\{ \frac{-118 - 41hL + e^{hL} (118 - 77hL)}{42h^2 L^2}, \frac{8(-1 + e^{hL})}{7hL}, \right. \\ \left. \frac{3(58 + 21hL + e^{hL}(-58 + 37hL))}{28h^2 L^2}, -\frac{12(-1 + e^{hL})}{7hL}, \frac{-286 - 239hL + e^{hL}(286 - 47hL)}{84h^2 L^2} \right\}$$

$$b_1, b_2, b_3, b_4, b_5, b_6 = \left\{ \frac{7(-1620 - 626hL - 73h^2 L^2 + e^{hL}(1620 - 994hL + 257h^2 L^2))}{2700h^3 L^3}, \right. \\ 0, \frac{900 + 1834hL + 287h^2 L^2 + e^{hL}(-900 - 934hL + 1097h^2 L^2)}{1350h^3 L^3}, \\ \left. \frac{2(810 + 412hL + 56h^2 L^2 + e^{hL}(-810 + 398hL - 49h^2 L^2))}{225h^3 L^3} \right\}$$

$$\begin{aligned}
& \frac{540 - 1226 h L - 1183 h^2 L^2 + e^{hL} (-540 + 1766 h L - 313 h^2 L^2)}{1350 h^3 L^3}, \\
& \left. \frac{-10980 - 6722 h L - 1741 h^2 L^2 + e^{hL} (10980 - 4258 h L + 509 h^2 L^2)}{2700 h^3 L^3} \right\} \\
c: & \left\{ 0, \frac{1}{4}, \frac{1}{4}, \frac{1}{2}, \frac{3}{4}, 1, 1 \right\}
\end{aligned}$$

References

- [1] S. Cox, P. Matthews, Exponential time differencing for stiff systems, *J. Comput. Phys.* 176 (2002) 430–455.
- [2] U.M. Ascher, S.J. Ruuth, B.T. Wetton, Implicit–explicit methods for time-dependent partial differential equations, *SIAM J. Numer. Anal.* 32 (1995) 797–823.
- [3] J.D. Lawson, Generalized Runge–Kutta processes for stable systems with large Lipschitz constants, *SIAM J. Numer. Anal.* 4 (1967) 372–380.
- [4] T.A. Driscoll, A composite Runge–Kutta method for the spectral solution of semilinear PDEs, *J. Comput. Phys.* 182 (2002) 357–367.
- [5] A.-K. Kassam, L.N. Trefethen, Fourth-order time-stepping for stiff PDEs, *SIAM J. Sci. Comput.* 26 (2005) 1214–1233.
- [6] S. Krogstad, Generalized integrating factor methods for stiff PDEs, *J. Comput. Phys.* 203 (2005) 72–88.
- [7] M. Hochbruck, A. Ostermann, Explicit exponential Runge–Kutta methods for semilinear parabolic problems, *SIAM J. Numer. Anal.* 43 (2005) 1069–1090.
- [8] B.V. Minchev, W.M. Wright, A review of exponential integrators for first order semi-linear problems, preprint, NTNU Trondheim, 2005.
- [9] M. Hochbruck, A. Ostermann, Exponential integrators, *Acta Numer.* 19 (2010) 209–286.
- [10] A. Couairon, A. Mysyrowicz, Femtosecond filamentation in transparent media, *Phys. Rep.* 441 (2007) 47–189.
- [11] H. Yang, A. Przekwas, A comparative study of advanced shock-capturing schemes applied to Burgers' equation, *J. Comput. Phys.* 102 (1992) 139–159.
- [12] M. Hochbruck, A. Ostermann, J. Schweitzer, Exponential Rosenbrock-type methods, *SIAM J. Numer. Anal.* 47 (2009) 786–803.
- [13] H. Berland, B. Skaflestad, W.M. Wright, EXPINT—a MATLAB package for exponential integrators, *ACM Trans. Math. Softw.* 33 (2007) 4.
- [14] B. Skaflestad, W. Wright, The scaling and modified squaring method for matrix functions related to the exponential, *Appl. Numer. Math.* 59 (2009) 783–799, selected papers from NUMDIFF-11.
- [15] M. Hochbruck, C. Lubich, H. Selhofer, Exponential integrators for large systems of differential equations, *SIAM J. Sci. Comput.* 19 (1998) 1552–1574.
- [16] M. Tokman, Efficient integration of large stiff systems of ODEs with exponential propagation iterative (EPI) methods, *J. Comput. Phys.* 213 (2006) 748–776.
- [17] Y. Saad, Analysis of some Krylov subspace approximations to the matrix exponential operator, *SIAM J. Numer. Anal.* 29 (1992) 209–228.
- [18] M. Hochbruck, C. Lubich, On Krylov subspace approximations to the matrix exponential operator, *SIAM J. Numer. Anal.* 34 (1997) 1911–1925.
- [19] J.C. Butcher, Coefficients for the study of Runge–Kutta integration processes, *J. Aust. Math. Soc.* 3 (1963) 185–201.
- [20] E. Fehlberg, Klassische Runge–Kutta-Formeln vierter und niedrigerer Ordnung mit Schrittweiten-Kontrolle und ihre Anwendung auf Wärmeleitungsprobleme, *Computing* 6 (1970) 61–71.
- [21] H. Berland, B. Owren, B. Skaflestad, B-series and order conditions for exponential integrators, *SIAM J. Numer. Anal.* 43 (2005) 1715–1727.
- [22] Wolfram Research, Inc., *Mathematica*, 2012.
- [23] J. Dormand, P. Prince, A family of embedded Runge–Kutta formulae, *J. Comput. Appl. Math.* 6 (1980) 19–26.
- [24] W.H. Enright, Analysis of error control strategies for continuous Runge–Kutta methods, *SIAM J. Numer. Anal.* 26 (1989) 588–599.
- [25] J. Shen, T. Tang, L.-L. Wang, *Spectral Methods: Algorithms, Analysis and Applications*, vol. 41, Springer, 2011.
- [26] A. Couairon, E. Gaižauskas, D. Faccio, A. Dubietis, P. Di Trapani, Nonlinear x-wave formation by femtosecond filamentation in Kerr media, *Phys. Rev. E* 73 (2006) 016608.
- [27] H.F. Johnson, An improved method for computing a discrete Hankel transform, *Comput. Phys. Commun.* 43 (1987) 181–202.
- [28] J. Marburger, Self-focusing: theory, *Prog. Quantum Electron.* 4 (1975) 35–110.
- [29] G. Beylkin, J.M. Keiser, L. Vozovoi, A new class of time discretization schemes for the solution of nonlinear PDEs, *J. Comput. Phys.* 147 (1998) 362–387.
- [30] G.H. Golub, C.F. Van Loan, *Matrix Computations*, vol. 3, JHU Press, 2012.
- [31] C. Moler, C. Van Loan, Nineteen dubious ways to compute the exponential of a matrix, twenty-five years later, *SIAM Rev.* 45 (2003) 3–49.
- [32] G. Guennebaud, B. Jacob, et al., *Eigen v3*, <http://eigen.tuxfamily.org>, 2010.
- [33] N.J. Higham, The scaling and squaring method for the matrix exponential revisited, *SIAM Rev.* 51 (2009) 747–764.
- [34] L.N. Trefethen, Evaluating matrix functions for exponential integrators via Carathéodory–Fejér approximation and contour integrals, *Electron. Trans. Numer. Anal.* 29 (2007) 1–18.
- [35] M. Tokman, J. Loffeld, Efficient design of exponential-Krylov integrators for large scale computing, *Proc. Comput. Sci.* 1 (2010) 229–237.